UNIVERSIDAD DE CUNDINAMARCA

Especialización en Inteligencia Artificial

CADI · Deep Learning - Conceptos

REDES NEURONALES II

Documento técnico · Perceptrón, backpropagation, funciones de activación y clasificación binaria y multiclase con TensorFlow + Keras y PyTorch

Actividad	R1-A2-S4 · Redes Neuronales II
Materia	604013/DEEP LEARNING -CONCEPTOS/FU/ CAD2202023205/EIAIIPA2026
Estudiante	**Yohann Javier Nikolay Salazar Cardenas**
Docente	Nidia Stella Garcia Roa
Periodo	2026
Fecha de entrega	10 de octubre de 2026
Entorno de desarrollo	Google Colaboratory · Python 3.11
Repositorio GitHub	`https://github.com/Yohann0011/Redes-Neuronales-II`

Repositorio oficial de la actividad: `https://github.com/Yohann0011/Redes-Neuronales-II` . Antes de la entrega, publique el contenido de la carpeta `Redes-Neuronales-II/` en ese repositorio con visibilidad pública y confirme que el enlace abra correctamente en una ventana de navegación privada.

Trabajo individual · Seis notebooks ejecutados en Google Colab · Sin redes convolucionales · TensorFlow + Keras y PyTorch

Tabla de contenido

1. Introducción

1.1 Objetivos de la actividad

1.2 Alcance y restricciones

1.3 Ruta de trabajo

2. Marco conceptual

2.1 Aprendizaje supervisado

2.2 La neurona artificial y forward propagation

2.3 Funciones de activación

2.4 Funciones de pérdida

2.5 Backpropagation y descenso de gradiente

2.6 Clasificación binaria vs clasificación multiclase

2.7 TensorFlow, Keras y PyTorch

2.8 El dataset MNIST

3. Entorno de trabajo y metodología

4. Práctica 1 · Perceptrón (código fuente y resultados)

5. Práctica 2 · Red de una capa y red multicapa con backpropagation

6. Práctica 3 · Clasificación binaria con TensorFlow + Keras

7. Práctica 4 · Clasificación multiclase con TensorFlow + Keras

8. Práctica 5 · MNIST multiclase con TensorFlow + Keras

9. Práctica 6 · MNIST multiclase con PyTorch

10. Comparación de enfoques y análisis global de resultados

11. Evidencias de ejecución

12. Repositorio GitHub y condiciones de entrega

13. Conclusiones

14. Referencias bibliográficas

Anexo A · Trazabilidad con la herramienta de recolección de datos

1. Introducción

En la actividad anterior (Redes Neuronales I) se construyeron redes neuronales cuyos pesos eran definidos manualmente: la red calculaba, pero no aprendía. El propósito de **Redes Neuronales II** es dar el paso siguiente y dotar al modelo de la capacidad de **corregir sus propios parámetros a partir del error que comete**.

Para lograrlo se integran seis piezas fundamentales del deep learning: las **funciones de activación** (Sigmoide, ReLU y Softmax), una **función de pérdida** que cuantifica el error, el algoritmo de **backpropagation** que determina cómo debe cambiar cada parámetro, el **descenso de gradiente** que aplica ese cambio, el **entrenamiento supervisado** que aporta las etiquetas verdaderas y, finalmente, los **frameworks** que automatizan todo el proceso a escala industrial.

El documento sigue una progresión deliberada: se parte de un problema intuitivo y de baja dimensionalidad (clasificar manzanas y bananos con tres características), se implementa desde cero con NumPy para comprender el mecanismo, y solo entonces se automatiza con TensorFlow + Keras y PyTorch, hasta llegar a un problema real de visión por computador: el reconocimiento de dígitos escritos a mano del dataset **MNIST** con diez clases.

> **Idea central de la actividad:** una red neuronal no aprende por el solo hecho de producir salidas; aprende cuando el error de esas salidas se utiliza para modificar sus parámetros. Todo lo demás — frameworks, optimizadores, arquitecturas— son formas cada vez más eficientes de ejecutar ese mismo ciclo.

1.1 Objetivos de la actividad

Objetivo general. Implementar, entrenar y evaluar redes neuronales artificiales (perceptrón, red de una capa y red multicapa) aplicando funciones de activación y el algoritmo de backpropagation, para resolver problemas de clasificación binaria y multiclase con NumPy, TensorFlow + Keras y PyTorch, documentando los resultados en un documento técnico reproducible.

Objetivos específicos.

1. Implementar el perceptrón con función de activación escalón y su regla de aprendizaje.
2. Implementar Sigmoide y ReLU, y aplicar backpropagation en una red de una capa y en una red multicapa construidas desde cero con NumPy.
3. Analizar experimentalmente el desvanecimiento del gradiente comparando ReLU y Sigmoide en la capa oculta.
4. Resolver un problema de clasificación binaria con TensorFlow + Keras y verificar la equivalencia con la implementación manual.
5. Extender el modelo a clasificación multiclase con salida Softmax y evaluarlo con matriz de confusión.
6. Entrenar un perceptrón multicapa sobre MNIST con TensorFlow + Keras, sin capas convolucionales.
7. Replicar la misma arquitectura en PyTorch con un ciclo de entrenamiento explícito.
8. Documentar el código fuente, los resultados y las evidencias de ejecución, y publicarlos en un repositorio GitHub.

1.2 Alcance y restricciones

- **Desarrollo individual.**

- **Sin redes convolucionales:** todas las arquitecturas son densas (`Flatten` + `Dense` / `nn.Linear`). Los notebooks lo verifican de forma programática.
- **Uso obligatorio de TensorFlow + Keras y PyTorch** sobre el dataset MNIST.
- **Desarrollo en Google Colab**, con reproducibilidad garantizada mediante semillas fijas (`random_state=42`, `np.random.seed(42)`, `tf.random.set_seed(42)`, `torch.manual_seed(42)`).
- **Inclusión de resultados y evidencias de ejecución**, no solo del código.

1.3 Ruta de trabajo

Etapa	Práctica	Problema	Aprendizaje central
1	Perceptrón con activación escalón	Manzana vs banano (NumPy)	Regla de aprendizaje, convergencia, límite lineal
2	Red de una capa y red multicapa	Manzana vs banano (NumPy)	Sigmoide, ReLU, BCE, backpropagation, desvanecimiento del gradiente
3	Clasificación binaria con framework	Manzana vs banano (Keras)	Automatización: `Sequential`, `compile`, `fit`
4	Clasificación multiclase	Manzana, banano y naranja (Keras)	Softmax, `sparse_categorical_crossentropy`, matriz de confusión
5	MNIST con TensorFlow + Keras	Dígitos 0-9 (10 clases)	Red densa 784 → 128 → 10, evaluación sobre 10 000 imágenes
6	MNIST con PyTorch	Dígitos 0-9 (10 clases)	Ciclo explícito: `zero_grad` → `backward` → `step`

2. Marco conceptual

2.1 Aprendizaje supervisado

En el aprendizaje supervisado cada ejemplo de entrenamiento incluye la **respuesta correcta**. Formalmente se dispone de un conjunto $D = \{(\mathbf{x}^{(k)}, y^{(k)})\}$ y el modelo busca una función $f(\mathbf{x}; \boldsymbol{\theta})$ cuyas predicciones se aproximen a las etiquetas verdaderas. La existencia de etiquetas es lo que permite medir el error y, por tanto, corregirlo: sin ellas no hay señal de aprendizaje.

En las prácticas de frutas, `X` contiene las características (peso, longitud, nivel de amarillo) e `y` contiene la clase correcta. En MNIST, `x` es la imagen de 28 × 28 píxeles e `y` es el dígito escrito.

2.2 La neurona artificial y forward propagation

Una neurona calcula una combinación lineal ponderada de sus entradas, añade un término de sesgo y aplica una función de activación no lineal:

$$z = \mathbf{w}^{\top}\mathbf{x} + b = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b \qquad a = g(z)$$

El **peso** w_i representa la influencia de la característica i-ésima; el **bias** b desplaza el umbral de activación. Forward propagation es el recorrido de los datos desde la capa de entrada hasta la capa de salida. En una red multicapa, la salida de cada capa es la entrada de la siguiente, lo que permite componer transformaciones no lineales.

2.3 Funciones de activación

Sin una función de activación no lineal, la composición de capas lineales seguiría siendo lineal: una red de diez capas colapsaría matemáticamente a una sola. Las activaciones son, por tanto, lo que otorga **capacidad de representación** al modelo.

Función	Expresión	Uso en esta actividad	Comportamiento y limitaciones
Escalón	0 si $z < 0$; 1 si $z \geq 0$	Perceptrón clásico (Práctica 1)	No es derivable en $z = 0$: incompatible con descenso de gradiente. Salida discreta, sin probabilidad.
Sigmoide	$\sigma(z) = 1 / (1 + e^{-z})$	Capa de salida binaria (Prácticas 2 y 3)	Interpretable como probabilidad en (0,1). Se satura en los extremos: $\sigma'(z) \leq 0.25$, lo que provoca desvanecimiento del gradiente en capas profundas.
ReLU	$\max(0, z)$	Capas ocultas (Prácticas 2 a 6)	Barata de evaluar, no se satura en la región positiva y su derivada vale 1. Riesgo de neuronas muertas si el bias es muy negativo.
Softmax	$e^{z_i} / \Sigma_j e^{z_j}$	Capa de salida multiclase (Prácticas 4, 5 y 6)	Convierte logits en una distribución de probabilidad que suma 1. La decisión se toma con argmax.

Derivadas utilizadas en backpropagation. La Sigmoide tiene la propiedad práctica $\sigma'(z) = \sigma(z)\cdot(1 - \sigma(z))$, es decir, su derivada se calcula a partir de su propia salida, que ya se obtuvo en forward. La derivada de ReLU es 1 si $z > 0$ y 0 en caso contrario. Estas dos expresiones son las que se implementan manualmente en la Práctica 2.

2.4 Funciones de pérdida

La pérdida resume en un único número qué tan equivocada está la red. Debe ser derivable respecto a los parámetros y decrecer a medida que el modelo mejora.

Pérdida	Expresión	Se usa cuando…	Lectura
Binary Cross-Entropy	$L = -(1/m)\ \Sigma\ [\ y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})\ ]$	Clasificación binaria con salida Sigmoide	Penaliza severamente las predicciones seguras y erróneas: si $y = 1$ y $\hat{y} \to 0$, la pérdida tiende a ∞.
Sparse Categorical Cross-Entropy	$L = -(1/m)\ \Sigma \log(\ \hat{y}_y\ )$	Multiclase con etiquetas enteras y salida Softmax	Solo interviene la probabilidad asignada a la clase verdadera. En el arranque vale $-\log(1/K)$.

> **Valores de referencia para interpretar la pérdida.** Con dos clases y predicciones de 0.5, la BCE vale $-\log(0.5) =$ **0.6931**. Con diez clases y probabilidades uniformes, la entropía cruzada vale $-\log(0.1) =$ **2.3026**. Ambos valores se verifican experimentalmente en los notebooks y constituyen el punto de partida esperado de un modelo sin entrenar: observarlos en la época 0 confirma que la implementación es correcta.

2.5 Backpropagation y descenso de gradiente

Backpropagation es la aplicación sistemática de la regla de la cadena para calcular el gradiente de la pérdida respecto a cada parámetro de la red, propagando la señal de error desde la capa de salida hacia las capas anteriores. No es un algoritmo de aprendizaje en sí mismo: es el procedimiento que obtiene los gradientes que luego consume el optimizador.

Para la red de una capa con Sigmoide y BCE, el resultado se simplifica notablemente:

$$\partial L/\partial z = \hat{y} - y \qquad \partial L/\partial \mathbf{w} = (1/m)\ \mathbf{X}^{\top}(\hat{y} - y) \qquad \partial L/\partial b = (1/m)\ \Sigma(\hat{y} - y)$$

Para una red multicapa, la señal se propaga hacia atrás atravesando cada activación:

$$\delta_2 = A_2 - y\ \rightarrow\ \partial L/\partial W_2 = {A_1}^{\top}\delta_2/m\ \rightarrow\ \delta_1 = (\delta_2\ {W_2}^{\top}) \odot g'(Z_1)\ \rightarrow\ \partial L/\partial W_1 = \mathbf{X}^{\top}\delta_1/m$$

El **descenso de gradiente** aplica entonces la actualización $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \partial L/\partial \boldsymbol{\theta}$, donde η es el learning rate: un hiperparámetro (no un peso) que controla el tamaño del paso. Un η demasiado grande hace oscilar la pérdida; uno demasiado pequeño ralentiza el aprendizaje.

> **Aclaración conceptual importante.** `dW` y `db` no son "el error": son cantidades que indican la dirección en la que deben modificarse los parámetros para reducir la pérdida. El error es la pérdida `L`; el gradiente es la respuesta a la pregunta "¿hacia dónde muevo cada peso y cuánto importa moverlo?".

2.6 Clasificación binaria vs clasificación multiclase

Aspecto	Clasificación binaria	Clasificación multiclase
Número de clases	2	3 o más (10 en MNIST)
Neuronas de salida	1	Una por clase
Activación de salida	Sigmoide	Softmax
Salida del modelo	P(clase 1) ∈ (0,1)	Vector de probabilidades que suma 1
Regla de decisión	$\hat{y} = 1$ si $p \geq 0.5$	$\hat{y} = \text{argmax}(p)$
Pérdida	binary_crossentropy	sparse_categorical_crossentropy
Evaluación adecuada	Accuracy, precisión, recall	Accuracy + matriz de confusión + reporte por clase
Práctica asociada	1, 2 y 3	4, 5 y 6

2.7 TensorFlow, Keras y PyTorch

TensorFlow es una biblioteca de cómputo numérico basada en grafos y tensores, con diferenciación automática y soporte de CPU, GPU y TPU. **Keras** es la API de alto nivel para construir y entrenar modelos: en su versión 3 es multi-backend y en Colab funciona sobre TensorFlow. **PyTorch** es un framework de grafos dinámicos cuyo ciclo de entrenamiento se escribe de forma explícita, lo que ofrece mayor control y transparencia.

Acción	NumPy manual	TensorFlow + Keras	PyTorch
Crear capas	Matrices `W` y `b`	`keras.layers.Dense`	`nn.Linear`
ReLU	`np.maximum(0, z)`	`activation="relu"`	`nn.ReLU()`
Sigmoide	Función propia	`activation="sigmoid"`	`nn.Sigmoid()`
Softmax	Cálculo manual	`activation="softmax"`	Implícito en `CrossEntropyLoss`
Pérdida	Función propia	`compile(loss=...)`	`nn.CrossEntropyLoss()`
Backpropagation	Derivadas escritas a mano	Automática dentro de `fit()`	`error.backward()`
Actualizar pesos	`W -= lr * dW`	Automática por el optimizador	`optimizador.step()`
Entrenar	`for` manual	`modelo.fit()`	`for` explícito
Evaluar	Cálculo manual	`modelo.evaluate()`	`eval()` + `no_grad()`

2.8 El dataset MNIST

MNIST (LeCun, Cortes y Burges, 2010) contiene **60 000 imágenes de entrenamiento** y **10 000 de prueba** de dígitos manuscritos en escala de grises, cada una de **28 × 28 píxeles**, centradas y normalizadas. Es el conjunto de referencia introductorio para clasificación multiclase de imágenes. Al aplanar cada imagen se obtiene un vector de **784 características**, lo que permite tratarla exactamente igual que el vector de tres características del problema de frutas: cambia la escala, no la lógica.

3. Entorno de trabajo y metodología

Las seis prácticas se desarrollaron como notebooks de **Google Colaboratory** y se ejecutaron de extremo a extremo. Cada notebook incluye celdas de Markdown con la explicación conceptual y celdas de código con la implementación, y **todas las salidas que aparecen en este documento son las salidas reales de esa ejecución**, no valores transcritos manualmente.

Componente	Versión / valor
Lenguaje	Python 3.11
Cálculo numérico	NumPy 2.4.6
Preprocesamiento y métricas	scikit-learn 1.9.1
Visualización	Matplotlib 3.11.2
Framework 1	TensorFlow 2.21.0 (CPU) · Keras 3.15.1, backend TensorFlow
Framework 2	PyTorch 2.14.1 · torchvision 0.29.1
Dataset	MNIST (60 000 / 10 000 imágenes de 28 × 28)
Semilla de reproducibilidad	42 en todas las prácticas
Dispositivo de ejecución	CPU, 2 núcleos, ~1 GB de RAM

Metodología aplicada en cada práctica

1. **Inspección de los datos** antes de modelar: formas, rangos, distribución de clases y visualización exploratoria.
2. **Partición entrenamiento / prueba** con `train_test_split`, estratificada y con semilla fija; las muestras de prueba nunca participan en el ajuste de parámetros.
3. **Preprocesamiento**: estandarización (`StandardScaler`) en frutas y normalización a [0,1] en MNIST. El escalador se ajusta solo con entrenamiento y se aplica a prueba.
4. **Implementación** del modelo, primero manual con NumPy y luego con framework.
5. **Entrenamiento** con registro de la pérdida por época.
6. **Evaluación** con pérdida, accuracy, matriz de confusión, reporte por clase y análisis de errores concretos.
7. **Verificación de la implementación**: comparación de los valores de pérdida inicial con las referencias teóricas (0.6931 en binario, 2.3026 en diez clases) y contraste entre modelos.

> **Nota técnica sobre la ejecución de MNIST en TensorFlow.** El entorno de ejecución disponible para esta actividad dispone de aproximadamente 1 GB de memoria, mientras que la sola importación de TensorFlow consume cerca de 620 MB. La llamada `modelo.fit(x_train_float32, ...)` materializa una copia `float32` completa del conjunto de entrenamiento (188 MB adicionales) y, sumada a los tensores internos del entrenamiento, provoca que el gestor de memoria termine el proceso del kernel. Por esta razón, la Práctica 5 entrena con un **ciclo explícito por lotes** usando `tf.GradientTape` y `optimizer.apply_gradients()`, convirtiendo y normalizando únicamente 128 imágenes por iteración. El consumo se mantiene plano en torno a 725 MB y el ciclo es matemáticamente idéntico al que ejecuta `fit()` internamente. La versión de alto nivel equivalente se incluye comentada en el notebook. En Colab con GPU este ajuste no es necesario.

4. Práctica 1 · Perceptrón (código fuente y resultados)

Notebook: `01_Perceptron_NumPy.ipynb` · **Problema:** clasificación binaria de manzanas y bananos con tres características (peso en gramos, longitud en centímetros y nivel de amarillo entre 0 y 1) · **Modelo:** perceptrón de Rosenblatt con activación escalón.

El perceptrón es la unidad de aprendizaje más simple del deep learning. Su regla de actualización solo modifica los parámetros cuando la predicción es incorrecta:

$$w_i \leftarrow w_i + \eta\,(y - \hat{y})\,x_i \qquad b \leftarrow b + \eta\,(y - \hat{y})$$

El teorema de convergencia garantiza que, si las clases son linealmente separables, encuentra un hiperplano separador en un número finito de épocas. Esta práctica constituye la **línea base** contra la cual se comparan los modelos siguientes.

4.1 Entorno, dataset y preprocesamiento

Se cargan las herramientas, se construye el dataset de 20 frutas y se realiza la partición estratificada 80/20 con estandarización de características.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import random

random.seed(42)
np.random.seed(42)
print("NumPy:", np.__version__)
print("Herramientas cargadas correctamente")
```

Salida de ejecución (evidencia)

```
NumPy: 2.4.6
Herramientas cargadas correctamente
```

```
X = np.array([
    # MANZANAS  (clase 0): [peso g, longitud cm, nivel de amarillo 0-1]
    [150, 7.5, 0.10], [160, 8.0, 0.15], [145, 7.2, 0.05],
    [170, 8.3, 0.20], [155, 7.8, 0.10], [165, 8.1, 0.15],
    [140, 7.0, 0.05], [175, 8.5, 0.20], [158, 7.9, 0.10],
    [162, 8.2, 0.15],

    # BANANOS   (clase 1)
    [120, 18.0, 0.90], [125, 19.0, 0.95], [115, 17.5, 0.85],
    [130, 20.0, 1.00], [118, 18.5, 0.90], [123, 19.5, 0.95],
    [110, 17.0, 0.80], [128, 20.5, 1.00], [121, 18.8, 0.90],
    [126, 19.2, 0.95]
], dtype=float)
```

```
y = np.array([0] * 10 + [1] * 10)

print("Matriz de características X:", X.shape, "-> (20 frutas, 3 características)")
print("Vector de etiquetas   y:", y.shape, "-> (0 = manzana, 1 = banano)")
print()
print("Primeras 3 manzanas:\n", X[:3])
print("Primeros 3 bananos:\n", X[10:13])
```

Salida de ejecución (evidencia)

```
Matriz de características X: (20, 3) -> (20 frutas, 3 características)
Vector de etiquetas   y: (20,) -> (0 = manzana, 1 = banano)

Primeras 3 manzanas:
 [[1.50e+02 7.50e+00 1.00e-01]
 [1.60e+02 8.00e+00 1.50e-01]
 [1.45e+02 7.20e+00 5.00e-02]]
Primeros 3 bananos:
 [[120.    18.     0.9 ]
 [125.    19.     0.95]
 [115.    17.5    0.85]]
```

4.2 Exploración visual de los datos

Antes de modelar se verifica gráficamente que las dos clases ocupen regiones distintas del espacio de características. Esta inspección es la que justifica esperar la convergencia del perceptrón.

4.3 Función escalón, inicialización y primera predicción

```
def step(z):
    """Función de activación escalón (umbral en 0)."""
    return (z >= 0).astype(int)


def predecir(X_datos, w, b):
    """Forward propagation de una sola neurona: z = X·w + b  ->  y_hat = step(z)."""
    z = X_datos @ w + b
    return step(z)


# --- Inicialización de parámetros ---
np.random.seed(42)
w = np.random.randn(3) * 0.01     # pesos pequeños aleatorios
b = 0.0                           # bias inicial

print("Pesos iniciales w =", np.round(w, 6))
print("Bias inicial    b =", round(b, 6))
print()
print("Predicciones iniciales (sin entrenar):", predecir(X_train, w, b))
print("Etiquetas reales                      :", y_train)
print("Aciertos iniciales:", int((predecir(X_train, w, b) == y_train).sum()), "de", len(y_train))
```

Salida de ejecución (evidencia)

```
Pesos iniciales w = [ 0.004967 -0.001383  0.006477]
Bias inicial    b = 0.0

Predicciones iniciales (sin entrenar): [1 1 0 0 1 1 0 0 0 1 1 0 1 0 1 1]
Etiquetas reales                     : [0 0 1 1 1 1 1 0 0 1 1 0 1 0 0 0]
Aciertos iniciales: 9 de 16
```

4.4 Una iteración de aprendizaje, paso a paso

Se ejecuta una sola pasada por el conjunto de entrenamiento imprimiendo cada corrección. Esto permite observar el mecanismo interno del aprendizaje supervisado.

```
np.random.seed(42)
w = np.random.randn(3) * 0.01
b = 0.0
eta = 0.1

print("=== ITERACIÓN 1 (paso a paso) ===")
print(f"{'#':>2} {'z':>9} {'y_hat':>5} {'y':>3} {'error':>6}   acción")
for k in range(len(X_train)):
    x_k, y_k = X_train[k], y_train[k]
    z_k = float(x_k @ w + b)
    y_hat = int(z_k >= 0)
    error = y_k - y_hat
    accion = "sin cambio (acierto)" if error == 0 else "se actualizan w y b"
    print(f"{k:>2} {z_k:>9.4f} {y_hat:>5} {y_k:>3} {error:>6}   {accion}")
    w = w + eta * error * x_k
    b = b + eta * error

print()
print("Pesos tras la iteración 1:", np.round(w, 6))
print("Bias  tras la iteración 1:", round(b, 6))
```

Salida de ejecución (evidencia)

```
=== ITERACIÓN 1 (paso a paso) ===
 #         z y_hat   y  error   acción
 0    0.0008     1   0     -1   se actualizan w y b
 1   -0.4165     0   0      0   sin cambio (acierto)
 2    0.1824     1   1      0   sin cambio (acierto)
 3    0.2019     1   1      0   sin cambio (acierto)
 4    0.1825     1   1      0   sin cambio (acierto)
 5    0.1827     1   1      0   sin cambio (acierto)
 6    0.1898     1   1      0   sin cambio (acierto)
 7   -0.3535     0   0      0   sin cambio (acierto)
 8   -0.3923     0   0      0   sin cambio (acierto)
 9    0.1801     1   1      0   sin cambio (acierto)
10    0.2022     1   1      0   sin cambio (acierto)
11   -0.3422     0   0      0   sin cambio (acierto)
12    0.2020     1   1      0   sin cambio (acierto)
13   -0.3769     0   0      0   sin cambio (acierto)
14   -0.3898     0   0      0   sin cambio (acierto)
15   -0.4545     0   0      0   sin cambio (acierto)

Pesos tras la iteración 1: [-0.105816  0.092776  0.099898]
Bias  tras la iteración 1: -0.1
```

4.5 Ciclo de entrenamiento completo

```
np.random.seed(42)
w = np.random.randn(3) * 0.01
b = 0.0
eta = 0.1
max_epochs = 100

historial_accuracy = []
epoch_convergencia = None

for epoch in range(max_epochs):
    actualizaciones = 0
    for k in range(len(X_train)):
        x_k, y_k = X_train[k], y_train[k]
        y_hat = int((x_k @ w + b) >= 0)
        error = y_k - y_hat
        if error != 0:
            w = w + eta * error * x_k
            b = b + eta * error
            actualizaciones += 1

    acc = float((predecir(X_train, w, b) == y_train).mean())
    historial_accuracy.append(acc)

    if epoch % 10 == 0 or actualizaciones == 0:
```

```
        print(f"Época {epoch:>3} | actualizaciones: {actualizaciones:>2} | accuracy entrenamiento: {acc:.
4f}")

    if actualizaciones == 0 and epoch_convergencia is None:
        epoch_convergencia = epoch
        print(f"\n>>> Convergió en la época {epoch}: ninguna predicción fue incorrecta.")
        break

print()
print("Pesos finales   w =", np.round(w, 4))
print("Bias final      b =", round(b, 4))
print("Épocas usadas     =", epoch + 1)
```

Salida de ejecución (evidencia)

```
Época   0 | actualizaciones:  1 | accuracy entrenamiento: 1.0000
Época   1 | actualizaciones:  0 | accuracy entrenamiento: 1.0000

>>> Convergió en la época 1: ninguna predicción fue incorrecta.

Pesos finales   w = [-0.1058  0.0928  0.0999]
Bias final      b = -0.1
Épocas usadas     = 2
```

4.6 Evaluación sobre datos no vistos

```
y_pred_test = predecir(X_test, w, b)
acc_test = accuracy_score(y_test, y_pred_test)

print("=== PREDICCIONES EN EL CONJUNTO DE PRUEBA ===")
print(f"{'Fruta':>6} {'Prob. z':>9} {'Predicción':>12} {'Real':>12} {'Resultado':>10}")
for i in range(len(X_test)):
    z_i = float(X_test[i] @ w + b)
    pred = "BANANO" if y_pred_test[i] == 1 else "MANZANA"
    real = "BANANO" if y_test[i] == 1 else "MANZANA"
    ok = "ACIERTO" if pred == real else "ERROR"
    print(f"{i+1:>6} {z_i:>9.4f} {pred:>12} {real:>12} {ok:>10}")

print()
print("Accuracy en prueba:", round(acc_test, 4))
print()
print("Matriz de confusión (filas = reales, columnas = predichas):")
print(confusion_matrix(y_test, y_pred_test))
print()
print(classification_report(y_test, y_pred_test, target_names=["Manzana", "Banano"], zero_division=0))
```

Salida de ejecución (evidencia)

```
=== PREDICCIONES EN EL CONJUNTO DE PRUEBA ===
 Fruta   Prob. z   Predicción         Real  Resultado
     1   -0.3171      MANZANA      MANZANA    ACIERTO
     2   -0.4295      MANZANA      MANZANA    ACIERTO
     3    0.1904       BANANO       BANANO    ACIERTO
     4    0.1984       BANANO       BANANO    ACIERTO

Accuracy en prueba: 1.0

Matriz de confusión (filas = reales, columnas = predichas):
[[2 0]
 [0 2]]

              precision    recall  f1-score   support

     Manzana       1.00      1.00      1.00         2
      Banano       1.00      1.00      1.00         2

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4
```

4.7 Frontera de decisión y pesos aprendidos

```
# Deshacer el escalado para graficar en unidades originales
media, desv = scaler.mean_, scaler.scale_

i_feat, j_feat = 1, 2      # longitud (cm) vs nivel de amarillo
rangos = np.array([X[:, i_feat].min(), X[:, i_feat].max(),
                   X[:, j_feat].min(), X[:, j_feat].max()])
margen = 0.1 * (rangos[1] - rangos[0] + rangos[3] - rangos[2])
g1 = np.linspace(rangos[0] - margen, rangos[1] + margen, 300)
g2 = np.linspace(rangos[2] - margen, rangos[3] + margen, 300)
G1, G2 = np.meshgrid(g1, g2)

# Se reconstruye el vector de 3 características usando la media para la tercera
puntos = np.column_stack([
    np.full(G1.size, media[0]),
    G1.ravel(),
    G2.ravel()
])
puntos_escalados = (puntos - media) / desv
Z_grid = (puntos_escalados @ w + b).reshape(G1.shape)

fig, axes = plt.subplots(1, 2, figsize=(13, 4.6))

# (a) Frontera de decisión
ax = axes[0]
ax.contourf(G1, G2, Z_grid, levels=20, cmap="RdYlGn", alpha=0.55)
ax.contour(G1, G2, Z_grid, levels=[0], colors="k", linewidths=2.2)
for clase, nombre, c in [(0, "Manzana", "#8e1b0e"), (1, "Banano", "#7d6608")]:
    m = y == clase
    ax.scatter(X[m, i_feat], X[m, j_feat], c=c, edgecolors="w", s=70, label=nombre)
ax.set_xlabel("Longitud (cm)"); ax.set_ylabel("Nivel de amarillo")
ax.set_title("Figura 2. Frontera de decisión lineal del perceptrón")
ax.legend()

# (b) Pesos aprendidos
ax = axes[1]
etiquetas = ["Peso (g)", "Longitud (cm)", "Nivel de amarillo"]
barras = ax.bar(etiquetas, w, color=["#3498db", "#2ecc71", "#e67e22"], edgecolor="k")
ax.axhline(0, color="k", lw=0.8)
ax.set_ylabel("Valor del peso")
ax.set_title("Figura 3. Pesos aprendidos por la neurona")
for barra, valor in zip(barras, w):
    ax.text(barra.get_x() + barra.get_width() / 2, valor,
            f"{valor:.3f}", ha="center",
            va="bottom" if valor >= 0 else "top", fontsize=9)
ax.grid(axis="x", alpha=0.3)

plt.tight_layout(); plt.savefig("fig02_perceptron_frontera.png", bbox_inches="tight"); plt.show()

print("Lectura de los pesos:")
print(" · La característica con mayor |peso| domina la decisión.")
print(f" · |w| = {np.round(np.abs(w), 4)}")
```

Salida de ejecución (evidencia)

```
Lectura de los pesos:
 · La característica con mayor |peso| domina la decisión.
 · |w| = [0.1058 0.0928 0.0999]
```

4.8 Predicción de una fruta nueva

```
fruta_nueva = np.array([[124.0, 19.0, 0.94]])      # liviana, alargada, muy amarilla
fruta_escalada = scaler.transform(fruta_nueva)

z_nueva = float((fruta_escalada @ w + b)[0])
pred_nueva = int(z_nueva >= 0)
nombre_pred = "BANANO" if pred_nueva == 1 else "MANZANA"

print("Fruta nueva (peso, longitud, amarillo):", fruta_nueva[0])
```

```
print("Valores escalados                    :", np.round(fruta_escalada[0], 4))
print("z = w·x + b                          :", round(z_nueva, 4))
print("step(z)                              :", pred_nueva)
print("La red piensa que es                 :", nombre_pred)
```

Salida de ejecución (evidencia)

```
Fruta nueva (peso, longitud, amarillo): [124.    19.     0.94]
Valores escalados                     : [-0.9271  0.9614  0.9859]
z = w·x + b                           : 0.1858
step(z)                               : 1
La red piensa que es                  : BANANO
```

4.9 Análisis de resultados de la Práctica 1

Indicador	Valor obtenido	Interpretación
Épocas hasta converger	2 (1 actualización efectiva)	El dataset es linealmente separable, por lo que basta un ajuste para separar las clases.
Aciertos en entrenamiento	16 de 16	Hiperplano separador encontrado.
Accuracy en prueba	1.0000 (4 de 4)	Generaliza sobre las frutas no vistas.
Pesos finales	w = [−0.1058, 0.0928, 0.0999], b = −0.1	El peso de mayor magnitud corresponde al peso en gramos, la característica con signo invertido.
Fruta nueva (124 g, 19 cm, 0.94)	z = 0.1858 → BANANO	Predicción correcta y coherente con las características.

Limitaciones identificadas. El perceptrón entrega una clase discreta y no una probabilidad: no puede expresar "estoy 92 % seguro". Aprende mediante una regla de corrección de errores caso por caso, no minimizando una función de pérdida, por lo que no admite descenso de gradiente (la función escalón no es derivable en $z = 0$). Es estrictamente lineal y un único ejemplo ruidoso puede desplazar el hiperplano indefinidamente. Estas cuatro limitaciones motivan directamente la Práctica 2.

5. Práctica 2 · Red de una capa y red multicapa con backpropagation

Notebook: `02_Una_Capa_y_Multicapa_NumPy_Backpropagation.ipynb` · **Problema:** el mismo dataset de frutas (clasificación binaria) · **Modelos:** red de una capa 3 → 1 con Sigmoide y red multicapa 3 → 4 → 1 con ReLU, ambas entrenadas con descenso de gradiente y backpropagation implementados manualmente.

Esta es la práctica conceptualmente central de la actividad: aquí se escribe a mano todo lo que los frameworks automatizan después. Se implementan Sigmoide, ReLU, la derivada de ReLU y la Binary Cross-Entropy, y se calculan explícitamente los gradientes de cada capa.

5.1 Funciones de activación y función de pérdida

Se implementan las tres piezas matemáticas y se verifican numéricamente. La comprobación de que la pérdida vale 0.6931 cuando el modelo predice 0.5 y crece hasta 4.6052 cuando predice 0.01 siendo la etiqueta 1 confirma que la Binary Cross-Entropy penaliza con severidad los errores confiados.

5.2 Red de una capa: inicialización y primera propagación

Con pesos iniciales casi nulos, $z \approx 0$ y $\sigma(0) = 0.5$: la red arranca sin opinión, que es el comportamiento esperado. La pérdida inicial de 0.6929 coincide con la referencia teórica $-\log(0.5) = 0.6931$.

```
m = len(X_train)

# 1) Forward
Z = X_train @ W + b
A = sigmoid(Z)

# 2) Loss
error = calcular_error(y_col, A)

# 3) Backpropagation (regla de la cadena)
dZ = A - y_col                      # dL/dZ  para BCE + Sigmoide
dW = X_train.T @ dZ / m             # dL/dW
db = np.mean(dZ)                    # dL/db

# 4) Descenso de gradiente
learning_rate = 0.1
W_nuevo = W - learning_rate * dW
b_nuevo = b - learning_rate * db

print("Pérdida antes de actualizar      :", round(float(error), 6))
print("dZ (primeros 5)                  :", np.round(dZ.ravel()[:5], 5))
print("dW = X^T dZ / m                  :", np.round(dW.ravel(), 6))
print("db                               :", round(float(db), 6))
print()
print("W antes:", np.round(W.ravel(), 6), "-> W después:", np.round(W_nuevo.ravel(), 6))
print("b antes:", round(float(b), 6), "        -> b después:", round(float(b_nuevo), 6))

# Verificación: la pérdida debe disminuir tras el paso
A2 = sigmoid(X_train @ W_nuevo + b_nuevo)
print()
print("Pérdida después de actualizar    :", round(float(calcular_error(y_col, A2)), 6))
print("¿Disminuyó?", calcular_error(y_col, A2) < error)
```

Salida de ejecución (evidencia)

```
Pérdida antes de actualizar      : 0.692932
dZ (primeros 5)                  : [ 0.50019  0.50039 -0.50025 -0.50041 -0.49978]
dW = X^T dZ / m                  : [ 0.466997 -0.497071 -0.497324]
db                               : 0.0

W antes: [ 0.004967 -0.001383  0.006477] -> W después: [-0.041733  0.048325  0.056209]
b antes: 0.0         -> b después: -0.0

Pérdida después de actualizar    : 0.624232
¿Disminuyó? True
```

5.3 Una vuelta de backpropagation, paso a paso

Se imprime cada cantidad intermedia del cálculo de gradientes y se verifica que la pérdida disminuye tras el paso de actualización (0.692932 → 0.624232). Esta verificación es la prueba empírica de que los gradientes están bien calculados.

```
np.random.seed(42)
W = np.random.randn(3, 1) * 0.01
b = 0.0
learning_rate = 0.1
epochs = 2000

historial_1capa = []
y_col = y_train.reshape(-1, 1)

for epoch in range(epochs):
    # 1. Forward propagation
    Z = X_train @ W + b
    A = sigmoid(Z)

    # 2. Loss
    error = float(calcular_error(y_col, A))
    historial_1capa.append(error)

    # 3. Backpropagation
    m = len(X_train)
    dZ = A - y_col
    dW = X_train.T @ dZ / m
    db = np.mean(dZ)

    # 4. Actualización de parámetros
    W = W - learning_rate * dW
    b = b - learning_rate * db

    if epoch % 400 == 0:
        print(f"Época {epoch:>5} | Loss {error:.6f}")

print(f"Época {epochs-1:>5} | Loss {historial_1capa[-1]:.6f}")
print()
print("Pesos finales W =", np.round(W.ravel(), 4))
print("Bias final    b =", round(float(b), 4))

W_1capa, b_1capa = W.copy(), float(b)
```

Salida de ejecución (evidencia)

```
Época     0 | Loss 0.692932
Época   400 | Loss 0.009274
Época   800 | Loss 0.004618
Época  1200 | Loss 0.003075
Época  1600 | Loss 0.002306
Época  1999 | Loss 0.001846

Pesos finales W = [-1.9852  2.2612  2.2839]
Bias final    b = -0.0272
```

5.4 Entrenamiento completo de la red de una capa (2000 épocas)

```
probabilidades = sigmoid(X_test @ W_1capa + b_1capa)
predicciones = (probabilidades >= 0.5).astype(int).ravel()
acc_1capa = accuracy_score(y_test, predicciones)

print("=== RED DE UNA CAPA (Sigmoide + BCE + backpropagation) ===")
print(f"{'Fruta':>6} {'P(banano)':>10} {'Predicción':>12} {'Real':>12} {'Resultado':>10}")
for i in range(len(X_test)):
    pred = "BANANO" if predicciones[i] == 1 else "MANZANA"
    real = "BANANO" if y_test[i] == 1 else "MANZANA"
    ok = "ACIERTO" if pred == real else "ERROR"
    print(f"{i+1:>6} {probabilidades[i,0]:>10.4f} {pred:>12} {real:>12} {ok:>10}")

print()
print("Accuracy en prueba:", round(acc_1capa, 4))
print("Matriz de confusión:\n", confusion_matrix(y_test, predicciones))
```

Salida de ejecución (evidencia)

```
=== RED DE UNA CAPA (Sigmoide + BCE + backpropagation) ===
 Fruta  P(banano)   Predicción         Real  Resultado
     1     0.0056      MANZANA      MANZANA    ACIERTO
     2     0.0009      MANZANA      MANZANA    ACIERTO
     3     0.9978       BANANO       BANANO    ACIERTO
     4     0.9979       BANANO       BANANO    ACIERTO

Accuracy en prueba: 1.0
Matriz de confusión:
 [[2 0]
 [0 2]]
```

5.5 Evaluación de la red de una capa

```
def inicializar_parametros(entrada, ocultas, salida, semilla=42):
    """Inicialización de He para capas con ReLU."""
    np.random.seed(semilla)
    W1 = np.random.randn(entrada, ocultas) * np.sqrt(2.0 / entrada)
    b1 = np.zeros((1, ocultas))
    W2 = np.random.randn(ocultas, salida) * np.sqrt(2.0 / ocultas)
    b2 = np.zeros((1, salida))
    return W1, b1, W2, b2


W1, b1, W2, b2 = inicializar_parametros(3, 4, 1)

print("W1:", W1.shape, "-> conecta 3 entradas con 4 neuronas ocultas")
print("b1:", b1.shape)
print("W2:", W2.shape, "-> conecta 4 neuronas ocultas con 1 salida")
print("b2:", b2.shape)
print()
print("W1 =\n", np.round(W1, 4))
print("W2 =", np.round(W2.ravel(), 4))
print()
print("Total de parámetros entrenables:", W1.size + b1.size + W2.size + b2.size)
```

Salida de ejecución (evidencia)

```
W1: (3, 4) -> conecta 3 entradas con 4 neuronas ocultas
b1: (1, 4)
W2: (4, 1) -> conecta 4 neuronas ocultas con 1 salida
b2: (1, 1)

W1 =
 [[ 0.4056 -0.1129  0.5288  1.2435]
 [-0.1912 -0.1912  1.2894  0.6266]
 [-0.3833  0.443  -0.3784 -0.3803]]
W2 = [ 0.1711 -1.3529 -1.2197 -0.3976]

Total de parámetros entrenables: 21
```

5.6 Red multicapa 3 → 4 → 1: inicialización de He

La inicialización de He, W ~ N(0, $\sqrt{(2/n_{ent})}$), preserva la varianza de la señal a través de capas ReLU y evita que los gradientes exploten o se desvanezcan desde la época 0. La red tiene exactamente 21 parámetros: W1 (3×4), b1 (4), W2 (4×1) y b2 (1).

```
m = len(X_train)

# ---- Forward ----
Z1 = X_train @ W1 + b1
A1 = relu(Z1)
Z2 = A1 @ W2 + b2
A2 = sigmoid(Z2)
error = calcular_error(y_col, A2)

# ---- Capa de salida ----
dZ2 = A2 - y_col
dW2 = A1.T @ dZ2 / m
db2 = np.mean(dZ2, axis=0, keepdims=True)

# ---- Propagación hacia la capa oculta ----
dA1 = dZ2 @ W2.T                        # el error se reparte según W2
dZ1 = dA1 * derivada_relu(Z1)           # se aplica la derivada de ReLU
dW1 = X_train.T @ dZ1 / m
db1 = np.mean(dZ1, axis=0, keepdims=True)

print("Pérdida actual                  :", round(float(error), 6))
print()
print("dZ2 (primeros 5)                :", np.round(dZ2.ravel()[:5], 5))
print("dW2 (gradientes capa de salida) :", np.round(dW2.ravel(), 6))
print("db2                             :", np.round(db2.ravel(), 6))
print()
print("dA1 (error que llega a la capa oculta), primeras 3 filas:\n", np.round(dA1[:3], 6))
print("dZ1 tras multiplicar por ReLU'(Z1), primeras 3 filas:\n", np.round(dZ1[:3], 6))
print("dW1 (gradientes capa oculta):\n", np.round(dW1, 6))
print("db1                             :", np.round(db1.ravel(), 6))
print()
print("Norma del gradiente |dW1| =", round(float(np.linalg.norm(dW1)), 6))
print("Norma del gradiente |dW2| =", round(float(np.linalg.norm(dW2)), 6))
```

Salida de ejecución (evidencia)

```
Pérdida actual                  : 0.952865

dZ2 (primeros 5)                : [ 0.42911  0.41368 -0.64392 -0.65825 -0.72563]
dW2 (gradientes capa de salida) : [ 0.208053 -0.128823 -0.153272  0.188029]
db2                             : [-0.13912]

dA1 (error que llega a la capa oculta), primeras 3 filas:
 [[ 0.073418 -0.580546 -0.523391 -0.170615]
 [ 0.070777 -0.559661 -0.504563 -0.164477]
 [-0.110171  0.871161  0.785396  0.256023]]
dZ1 tras multiplicar por ReLU'(Z1), primeras 3 filas:
 [[ 0.073418 -0.       -0.       -0.170615]
 [ 0.070777 -0.       -0.       -0.164477]
 [-0.        0.871161  0.785396  0.      ]]
dW1 (gradientes capa oculta):
 [[ 0.033361 -0.450459 -0.450928 -0.074916]
 [-0.03828   0.492116  0.465746  0.074185]
 [-0.038474  0.490994  0.462845  0.07385 ]]
db1                             : [ 0.038173  0.490061  0.416972 -0.075493]

Norma del gradiente |dW1| = 1.158135
Norma del gradiente |dW2| = 0.34457
```

5.7 Forward propagation multicapa

```
np.random.seed(42)
W1, b1, W2, b2 = inicializar_parametros(3, 4, 1, semilla=42)
learning_rate = 0.05
```

```
epochs = 3000
historial_relu = []

for epoch in range(epochs):
    # 1. Forward propagation
    Z1 = X_train @ W1 + b1
    A1 = relu(Z1)
    Z2 = A1 @ W2 + b2
    A2 = sigmoid(Z2)

    # 2. Loss
    error = float(calcular_error(y_col, A2))
    historial_relu.append(error)

    # 3. Backpropagation
    m = len(X_train)
    dZ2 = A2 - y_col
    dW2 = A1.T @ dZ2 / m
    db2 = np.mean(dZ2, axis=0, keepdims=True)

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * derivada_relu(Z1)
    dW1 = X_train.T @ dZ1 / m
    db1 = np.mean(dZ1, axis=0, keepdims=True)

    # 4. Actualización (descenso de gradiente)
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    if epoch % 500 == 0:
        print(f"Época {epoch:>5} | Loss {error:.6f}")

print(f"Época {epochs-1:>5} | Loss {historial_relu[-1]:.6f}")

W1_relu, b1_relu, W2_relu, b2_relu = W1.copy(), b1.copy(), W2.copy(), b2.copy()
print()
print("W1 final =\n", np.round(W1_relu, 4))
print("W2 final =", np.round(W2_relu.ravel(), 4))
```

Salida de ejecución (evidencia)

```
Época     0 | Loss 0.952865
Época   500 | Loss 0.055464
Época  1000 | Loss 0.025895
Época  1500 | Loss 0.016626
Época  2000 | Loss 0.012180
Época  2500 | Loss 0.009587
Época  2999 | Loss 0.007896

W1 final =
 [[ 0.4751 -0.017   1.174   1.652 ]
 [-0.3647 -0.2951  0.205  -0.1614]
 [-0.562   0.3391 -1.4779 -1.1871]]
W2 final = [-0.6199 -1.3377 -1.8684 -1.6722]
```

5.8 Backpropagation multicapa, paso a paso

La señal de corrección nace en la salida y se propaga hacia atrás. El término `dA1 * derivada_relu(Z1)` es el que anula el gradiente de las neuronas cuyo valor previo fue negativo: se observa directamente en la salida, donde varias columnas de dZ1 quedan en cero.

```
def forward_multicapa(X_datos, W1, b1, W2, b2, act_oculta=relu):
    """Propagación hacia adelante de la red 3 -> 4 -> 1."""
    Z1 = X_datos @ W1 + b1
    A1 = act_oculta(Z1)
    Z2 = A1 @ W2 + b2
    return sigmoid(Z2), A1, Z1
```

```
probs_multi, A1_test, Z1_test = forward_multicapa(X_test, W1_relu, b1_relu, W2_relu, b2_relu)
pred_multi = (probs_multi >= 0.5).astype(int).ravel()
acc_multi = accuracy_score(y_test, pred_multi)

print("=== RED MULTICAPA 3 -> 4 -> 1 (ReLU + Sigmoide) ===")
print(f"{'Fruta':>6} {'P(banano)':>10} {'Predicción':>12} {'Real':>12} {'Resultado':>10}")
for i in range(len(X_test)):
    pred = "BANANO" if pred_multi[i] == 1 else "MANZANA"
    real = "BANANO" if y_test[i] == 1 else "MANZANA"
    ok = "ACIERTO" if pred == real else "ERROR"
    print(f"{i+1:>6} {probs_multi[i,0]:>10.6f} {pred:>12} {real:>12} {ok:>10}")

print()
print("Accuracy en prueba:", round(acc_multi, 4))
print("Matriz de confusión:\n", confusion_matrix(y_test, pred_multi))
print()
print("Activaciones de la capa oculta en prueba (qué neurona responde a cada fruta):")
print(np.round(A1_test, 4))
```

Salida de ejecución (evidencia)

```
=== RED MULTICAPA 3 -> 4 -> 1 (ReLU + Sigmoide) ===
 Fruta  P(banano)   Predicción         Real  Resultado
     1   0.014244      MANZANA      MANZANA    ACIERTO
     2   0.000021      MANZANA      MANZANA    ACIERTO
     3   0.985087       BANANO       BANANO    ACIERTO
     4   0.985087       BANANO       BANANO    ACIERTO

Accuracy en prueba: 1.0
Matriz de confusión:
 [[2 0]
 [0 2]]

Activaciones de la capa oculta en prueba (qué neurona responde a cada fruta):
[[1.2102 0.     2.1271 2.2145]
 [1.6851 0.     3.5213 4.3988]
 [0.     0.     0.     0.    ]
 [0.     0.     0.     0.    ]]
```

5.9 Entrenamiento y evaluación de la red multicapa (3000 épocas)

```
def entrenar_multicapa(act_oculta, derivada_act, learning_rate, epochs, semilla=42):
    """Entrena la red 3->4->1 con la activación indicada y devuelve historial y parámetros."""
    W1, b1, W2, b2 = inicializar_parametros(3, 4, 1, semilla=semilla)
    historial = []
    for _ in range(epochs):
        Z1 = X_train @ W1 + b1
        A1 = act_oculta(Z1)
        Z2 = A1 @ W2 + b2
        A2 = sigmoid(Z2)
        historial.append(float(calcular_error(y_col, A2)))

        m = len(X_train)
        dZ2 = A2 - y_col
        dW2 = A1.T @ dZ2 / m
        db2 = np.mean(dZ2, axis=0, keepdims=True)
        dA1 = dZ2 @ W2.T
        dZ1 = dA1 * derivada_act(Z1)
        dW1 = X_train.T @ dZ1 / m
        db1 = np.mean(dZ1, axis=0, keepdims=True)

        W1 -= learning_rate * dW1
        b1 -= learning_rate * db1
        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2
    return historial, (W1, b1, W2, b2)


def derivada_sigmoid(z):
    s = sigmoid(z)
    return s * (1 - s)
```

```
hist_relu, params_relu = entrenar_multicapa(relu, derivada_relu, 0.05, 3000)
hist_sig, params_sig = entrenar_multicapa(sigmoid, derivada_sigmoid, 0.05, 3000)

print(f"{'Época':>7} {'Loss ReLU':>12} {'Loss Sigmoide':>15}")
for e in [0, 500, 1000, 1500, 2000, 2500, 2999]:
    print(f"{e:>7} {hist_relu[e]:>12.6f} {hist_sig[e]:>15.6f}")

print()
print("Pérdida final  ReLU    :", round(hist_relu[-1], 6))
print("Pérdida final  Sigmoide:", round(hist_sig[-1], 6))
```

Salida de ejecución (evidencia)

```
  Época    Loss ReLU   Loss Sigmoide
      0     0.952865        1.022297
    500     0.055464        0.114070
   1000     0.025895        0.050687
   1500     0.016626        0.029897
   2000     0.012180        0.020223
   2500     0.009587        0.014925
   2999     0.007896        0.011692

Pérdida final  ReLU    : 0.007896
Pérdida final  Sigmoide: 0.011692
```

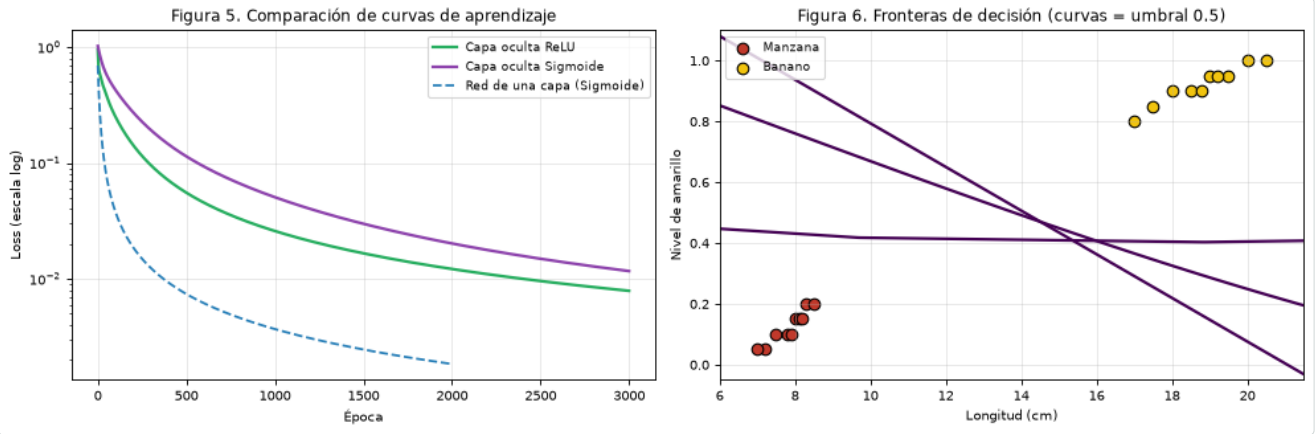


Figura 4. Comparación de curvas de aprendizaje (ReLU vs Sigmoide vs una capa) y fronteras de decisión.

```
# Norma del gradiente en la capa oculta al inicio del entrenamiento (misma semilla)
W1i, b1i, W2i, b2i = inicializar_parametros(3, 4, 1, semilla=42)
Z1 = X_train @ W1i + b1i
A1r, A1s = relu(Z1), sigmoid(Z1)
Z2r, Z2s = A1r @ W2i + b2i, A1s @ W2i + b2i
dZ2 = sigmoid(Z2r) - y_col
dA1 = dZ2 @ W2i.T

norma_relu = float(np.linalg.norm(X_train.T @ (dA1 * derivada_relu(Z1)) / len(X_train)))
norma_sig = float(np.linalg.norm(X_train.T @ (dA1 * derivada_sigmoid(Z1)) / len(X_train)))

print("Norma ||dW1|| con ReLU en la capa oculta    :", f"{norma_relu:.6e}")
print("Norma ||dW1|| con Sigmoide en la capa oculta:", f"{norma_sig:.6e}")
print("Razón ReLU / Sigmoide                       :", f"{norma_relu / norma_sig:.2f}x")
```

Salida de ejecución (evidencia)

```
Norma ||dW1|| con ReLU en la capa oculta    : 1.158135e+00
Norma ||dW1|| con Sigmoide en la capa oculta: 4.375875e-01
Razón ReLU / Sigmoide                       : 2.65x
```

```
fig, axes = plt.subplots(1, 2, figsize=(12.5, 4.2))

axes[0].plot(hist_relu, label="Capa oculta ReLU", color="#27ae60", lw=2)
```

```
axes[0].plot(hist_sig, label="Capa oculta Sigmoide", color="#8e44ad", lw=2)
axes[0].plot(historial_1capa, label="Red de una capa (Sigmoide)", color="#2980b9", lw=1.6, ls="--")
axes[0].set_yscale("log")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("Loss (escala log)")
axes[0].set_title("Figura 5. Comparación de curvas de aprendizaje")
axes[0].legend(fontsize=8)

# Fronteras de decisión de los tres modelos
media, desv = scaler.mean_, scaler.scale_
i_f, j_f = 1, 2
g1 = np.linspace(X[:, i_f].min() - 1, X[:, i_f].max() + 1, 300)
g2 = np.linspace(X[:, j_f].min() - 0.1, X[:, j_f].max() + 0.1, 300)
G1, G2 = np.meshgrid(g1, g2)
pts = np.column_stack([np.full(G1.size, media[0]), G1.ravel(), G2.ravel()])
pts_e = (pts - media) / desv

modelos = [("Una capa (Sigmoide)", lambda Xe: sigmoid(Xe @ W_1capa + b_1capa).ravel()),
           ("Multicapa ReLU", lambda Xe: forward_multicapa(Xe, *params_relu)[0].ravel()),
           ("Multicapa Sigmoide", lambda Xe: forward_multicapa(Xe, *params_sig, act_oculta=sigmoid)
[0].ravel())]

ax = axes[1]
for nombre, fn in modelos:
    Zg = fn(pts_e).reshape(G1.shape)
    ax.contour(G1, G2, Zg, levels=[0.5], linewidths=2)
for clase, nombre, c in [(0, "Manzana", "#c0392b"), (1, "Banano", "#f1c40f")]:
    msk = y == clase
    ax.scatter(X[msk, i_f], X[msk, j_f], c=c, edgecolors="k", s=60, label=nombre)
ax.set_xlabel("Longitud (cm)"); ax.set_ylabel("Nivel de amarillo")
ax.set_title("Figura 6. Fronteras de decisión (curvas = umbral 0.5)")
ax.legend(fontsize=8)

plt.tight_layout(); plt.savefig("fig04_comparacion_activaciones.png", bbox_inches="tight"); plt.show()
```

5.10 Experimento comparativo: ReLU vs Sigmoide en la capa oculta

Ambas redes comparten arquitectura, datos, semilla y épocas; solo cambia la activación de la capa oculta. El objetivo es observar el **desvanecimiento del gradiente**: como $\sigma'(z) \leq 0.25$, cada capa multiplica la señal de corrección por un factor pequeño.

5.11 Análisis de resultados de la Práctica 2

Modelo	Pérdida inicial → final	Loss en prueba	Accuracy	Observación
Una capa · Sigmoide (2000 épocas, η = 0.1)	0.692932 → 0.001846	0.002713	1.0000	Reducción de pérdida del 99.73 %. Probabilidades de prueba 0.0056, 0.0009, 0.9978 y 0.9979.
Multicapa · ReLU (3000 épocas, η = 0.05)	0.952865 → 0.007896	0.011104	1.0000	Converge más rápido en las primeras épocas que su equivalente con Sigmoide.
Multicapa · Sigmoide oculta (3000 épocas, η = 0.05)	1.022297 → 0.011692	0.013006	1.0000	Misma arquitectura y datos, pero aprendizaje más lento.

Evidencia del desvanecimiento del gradiente. Con los mismos parámetros iniciales, la norma del gradiente de la primera capa es **1.158135 con ReLU** frente a **0.437588 con Sigmoide**: una razón de **2.65×**. La Sigmoide comprime la señal de corrección porque su derivada nunca supera 0.25, mientras que ReLU la transmite íntegra en la región positiva. Este es el motivo por el cual ReLU es la activación estándar en capas ocultas y la Sigmoide se reserva para la salida binaria.

Lectura de la capa oculta. Las activaciones ReLU de las frutas de prueba muestran que las dos manzanas activan las neuronas 1, 3 y 4, mientras que los dos bananos no activan ninguna: la capa oculta ha aprendido una representación en la que las clases se separan por un patrón de activación distinto, no por un simple umbral sobre una característica.

> **Advertencia metodológica.** Un accuracy de 1.0 sobre cuatro frutas de prueba no demuestra que el modelo sea perfecto: el conjunto es pequeño (20 frutas) y deliberadamente separable con fines didácticos. La generalización real se valida en las Prácticas 5 y 6 sobre 10 000 imágenes de MNIST.

6. Práctica 3 · Clasificación binaria con TensorFlow + Keras

Notebook: `03_Frutas_Binario_Keras.ipynb` · **Problema:** manzana vs banano · **Modelo:** `Sequential` 3 → 4 → 1 (ReLU + Sigmoide), optimizador Adam con η = 0.01, pérdida `binary_crossentropy`, 200 épocas y batch de 4.

El objetivo no es cambiar la lógica matemática, sino comprobar que un framework automatiza exactamente las operaciones que se implementaron a mano en la Práctica 2. La arquitectura se mantiene idéntica (3 → 4 → 1) para que la comparación sea directa.

6.1 Entorno y versiones

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
import random

random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow:", tf.__version__)
print("Keras     :", keras.__version__)
print("Backend   :", keras.backend.backend())
print("GPU disponible:", bool(tf.config.list_physical_devices('GPU')))
```

Salida de ejecución (evidencia)

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791644414.323464   24058 port.cc:153] oneDNN custom operations are on. You may see slightly
different numerical results due to floating-point round-off errors from different computation orders. To turn
them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791644417.069815   24058 port.cc:153] oneDNN custom operations are on. You may see slightly
different numerical results due to floating-point round-off errors from different computation orders. To turn
them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
TensorFlow: 2.21.0
Keras     : 3.15.1
Backend   : tensorflow
GPU disponible: False
```

6.2 Datos y formato de etiquetas

Se reutiliza el dataset, la partición y el escalado de la Práctica 2. Keras espera las etiquetas de clasificación binaria como matriz columna de forma (m, 1).

```
X = np.array([
    # MANZANAS  (clase 0): [peso g, longitud cm, nivel de amarillo 0-1]
    [150, 7.5, 0.10], [160, 8.0, 0.15], [145, 7.2, 0.05],
    [170, 8.3, 0.20], [155, 7.8, 0.10], [165, 8.1, 0.15],
    [140, 7.0, 0.05], [175, 8.5, 0.20], [158, 7.9, 0.10],
    [162, 8.2, 0.15],

    # BANANOS   (clase 1)
    [120, 18.0, 0.90], [125, 19.0, 0.95], [115, 17.5, 0.85],
    [130, 20.0, 1.00], [118, 18.5, 0.90], [123, 19.5, 0.95],
    [110, 17.0, 0.80], [128, 20.5, 1.00], [121, 18.8, 0.90],
    [126, 19.2, 0.95]
], dtype=float)

y = np.array([0] * 10 + [1] * 10)

print("Matriz de características X:", X.shape, "-> (20 frutas, 3 características)")
print("Vector de etiquetas   y:", y.shape, "-> (0 = manzana, 1 = banano)")
print()
print("Primeras 3 manzanas:\n", X[:3])
print("Primeros 3 bananos:\n", X[10:13])
```

Salida de ejecución (evidencia)

```
Matriz de características X: (20, 3) -> (20 frutas, 3 características)
Vector de etiquetas   y: (20,) -> (0 = manzana, 1 = banano)

Primeras 3 manzanas:
 [[1.50e+02 7.50e+00 1.00e-01]
 [1.60e+02 8.00e+00 1.50e-01]
 [1.45e+02 7.20e+00 5.00e-02]]
Primeros 3 bananos:
 [[120.    18.     0.9 ]
 [125.    19.     0.95]
 [115.    17.5    0.85]]
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 80 % entrenamiento / 20 % prueba, conservando la proporción de clases
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# El escalado se AJUSTA solo con entrenamiento y se APLICA a prueba
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("X_train:", X_train.shape, "| X_test:", X_test.shape)
print("y_train:", y_train.shape, "| y_test:", y_test.shape)
print("Distribución de clases en prueba:", np.bincount(y_test))
print()
print("Media de cada característica en entrenamiento (debe ser ~0):")
print(np.round(X_train.mean(axis=0), 6))
print("Desviación estándar (debe ser ~1):")
print(np.round(X_train.std(axis=0), 6))
```

Salida de ejecución (evidencia)

```
X_train: (16, 3) | X_test: (4, 3)
y_train: (16,) | y_test: (4,)
Distribución de clases en prueba: [2 2]

Media de cada característica en entrenamiento (debe ser ~0):
[0. 0. 0.]
```

```
Desviación estándar (debe ser ~1):
[1. 1. 1.]
```

```
# Keras espera las etiquetas de clasificación binaria como matriz columna (m, 1)
y_train_k = y_train.reshape(-1, 1).astype("float32")
y_test_k = y_test.reshape(-1, 1).astype("float32")

print("X_train:", X_train.shape, X_train.dtype)
print("y_train:", y_train_k.shape, y_train_k.dtype)
print("X_test :", X_test.shape, "| y_test:", y_test_k.shape)
```

Salida de ejecución (evidencia)

```
X_train: (16, 3) float64
y_train: (16, 1) float32
X_test : (4, 3) | y_test: (4, 1)
```

6.3 Construcción del modelo y verificación de parámetros

Se usa siempre `keras.layers.Dense(...)`. Escribir `Dense(...)` sin importarlo provoca `NameError: name 'Dense' is not defined`, uno de los errores documentados en la sección 6.6.

```
tf.random.set_seed(42)

modelo = keras.Sequential([
    keras.layers.Input(shape=(3,)),
    keras.layers.Dense(4, activation="relu", name="oculta_relu"),
    keras.layers.Dense(1, activation="sigmoid", name="salida_sigmoid")
])

modelo.summary()
```

```
total_params = modelo.count_params()
print("Parámetros entrenables declarados por Keras:", total_params)
print("Parámetros de la implementación NumPy       :", 3*4 + 4 + 4*1 + 1)
print("Coinciden:", total_params == (3*4 + 4 + 4*1 + 1))
```

Salida de ejecución (evidencia)

```
Parámetros entrenables declarados por Keras: 21
Parámetros de la implementación NumPy       : 21
Coinciden: True
```

6.4 Compilación

Compilar significa elegir tres piezas del aprendizaje: el optimizador (Adam), la pérdida (binary_crossentropy) y la métrica de reporte (accuracy). La métrica **no** guía el aprendizaje: solo se informa.

```
modelo.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.01),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

print("Optimizador:", modelo.optimizer.__class__.__name__)
print("Pérdida    :", modelo.loss)
print("Métricas   :", modelo.metrics_names)
```

Salida de ejecución (evidencia)

```
Optimizador: Adam
Pérdida    : binary_crossentropy
Métricas   : ['loss', 'compile_metrics']
```

6.5 Entrenamiento, evaluación y pesos aprendidos

```
historial = modelo.fit(
    X_train,
    y_train_k,
    epochs=200,
    batch_size=4,
    verbose=0
)

print("Error inicial (época 1)  :", round(historial.history["loss"][0], 6))
print("Error final   (época 200):", round(historial.history["loss"][-1], 6))
print()
print(f"{'Época':>7}{'Loss':>12}{'Accuracy':>11}")
for e in [0, 24, 49, 99, 149, 199]:
    print(f"{e+1:>7}{historial.history['loss'][e]:>12.6f}{historial.history['accuracy'][e]:>11.4f}")
```

Salida de ejecución (evidencia)

```
Error inicial (época 1)  : 1.392065
Error final   (época 200): 0.01708

  Época        Loss   Accuracy
      1    1.392065     0.5000
     25    0.205516     1.0000
     50    0.112124     1.0000
    100    0.047774     1.0000
    150    0.026606     1.0000
    200    0.017080     1.0000
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 3.8))

axes[0].plot(historial.history["loss"], color="#c0392b", lw=2)
axes[0].set_title("Figura 7. Pérdida durante el entrenamiento (Keras)")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("binary_crossentropy")
axes[0].set_yscale("log")

axes[1].plot(historial.history["accuracy"], color="#2980b9", lw=2)
axes[1].set_title("Figura 8. Accuracy durante el entrenamiento")
axes[1].set_xlabel("Época"); axes[1].set_ylabel("Accuracy")
axes[1].set_ylim(0.4, 1.05)

plt.tight_layout(); plt.savefig("fig05_keras_frutas_binario.png", bbox_inches="tight"); plt.show()
```

```
test_loss, test_accuracy = modelo.evaluate(X_test, y_test_k, verbose=0)

print("=== RESULTADOS EN EL CONJUNTO DE PRUEBA ===")
print("Error (binary_crossentropy):", round(float(test_loss), 6))
print("Accuracy                   :", round(float(test_accuracy), 4))
print()

probabilidades = modelo.predict(X_test, verbose=0)
predicciones = (probabilidades >= 0.5).astype(int).ravel()

print(f"{'Fruta':>6}{'P(banano)':>12}{'Predicción':>13}{'Real':>12}{'Resultado':>11}")
for i in range(len(X_test)):
    pred = "BANANO" if predicciones[i] == 1 else "MANZANA"
    real = "BANANO" if y_test[i] == 1 else "MANZANA"
    ok = "ACIERTO" if pred == real else "ERROR"
    print(f"{i+1:>6}{probabilidades[i,0]:>12.6f}{pred:>13}{real:>12}{ok:>11}")
```

Salida de ejecución (evidencia)

```
=== RESULTADOS EN EL CONJUNTO DE PRUEBA ===
Error (binary_crossentropy): 0.017033
Accuracy                   : 1.0

 Fruta   P(banano)   Predicción        Real  Resultado
     1    0.033234      MANZANA     MANZANA    ACIERTO
     2    0.033234      MANZANA     MANZANA    ACIERTO
```

```
     3    0.999720       BANANO      BANANO    ACIERTO
     4    0.999746       BANANO      BANANO    ACIERTO
```

```
fruta_nueva = np.array([[124.0, 19.0, 0.94]])
fruta_nueva_escalada = scaler.transform(fruta_nueva)

probabilidad = modelo.predict(fruta_nueva_escalada, verbose=0)
resultado = "BANANO" if probabilidad[0][0] >= 0.5 else "MANZANA"

print("Fruta nueva (peso, longitud, amarillo):", fruta_nueva[0])
print("Valores escalados                      :", np.round(fruta_nueva_escalada[0], 4))
print("Probabilidad de ser banano             :", round(float(probabilidad[0][0]), 6))
print("La red piensa que es                   :", resultado)
```

Salida de ejecución (evidencia)

```
Fruta nueva (peso, longitud, amarillo): [124.    19.     0.94]
Valores escalados                      : [-0.9271  0.9614  0.9859]
Probabilidad de ser banano             : 0.999793
La red piensa que es                   : BANANO
```

```
W1_k, b1_k = modelo.get_layer("oculta_relu").get_weights()
W2_k, b2_k = modelo.get_layer("salida_sigmoid").get_weights()

print("Pesos de la capa oculta W1 (3x4):")
print(np.round(W1_k, 4))
print("Bias de la capa oculta b1:", np.round(b1_k.ravel(), 4))
print()
print("Pesos de la salida W2 (4x1):", np.round(W2_k.ravel(), 4))
print("Bias de la salida  b2       :", round(float(b2_k.ravel()[0]), 4))
print()
print("Activaciones ReLU de la capa oculta para las frutas de prueba:")


def relu(z):
    return np.maximum(0.0, z)


print(np.round(relu(X_test @ W1_k + b1_k), 4))
print("-> Las neuronas en 0 están inactivas para esa fruta; las positivas aportan evidencia.")
```

Salida de ejecución (evidencia)

```
Pesos de la capa oculta W1 (3x4):
[[ 0.2053 -0.9923 -0.9678  0.3109]
 [-0.3664  1.6038  1.5694 -0.1699]
 [ 0.455   1.5742 -0.0654  0.3394]]
Bias de la capa oculta b1: [-0.4214  1.3422  0.7045 -0.4553]

Pesos de la salida W2 (4x1): [0.7238 1.3785 1.4679 0.1543]
Bias de la salida  b2       : -3.3704

Activaciones ReLU de la capa oculta para las frutas de prueba:
[[0.     0.     0.     0.    ]
 [0.     0.     0.     0.    ]
 [0.     5.066  3.1121 0.    ]
 [0.     4.9991 3.2409 0.    ]]
-> Las neuronas en 0 están inactivas para esa fruta; las positivas aportan evidencia.
```

6.6 Análisis de resultados y errores frecuentes

Indicador	Valor obtenido	Interpretación
Parámetros entrenables	21 (16 en la capa oculta + 5 en la salida)	Coincide exactamente con la implementación NumPy de la Práctica 2.
Pérdida inicial → final	1.392065 → 0.017080 (200 épocas)	El modelo aprende el patrón.
Accuracy en entrenamiento	1.0000 desde la época 25	Convergencia rápida.
Loss / Accuracy en prueba	0.017033 / 1.0000	Clasifica correctamente las cuatro frutas no vistas.
Probabilidades de prueba	0.0332, 0.0332, 0.9997, 0.9997	Mayor margen que el modelo NumPy.
Fruta nueva (124 g, 19 cm, 0.94)	P(banano) = 0.999793 → BANANO	Coincide con la predicción del perceptrón y de la red NumPy.

Interpretación de los pesos aprendidos. El bias de la capa de salida es −3.3704: la red necesita una activación oculta suficientemente grande para superar ese umbral y declarar "banano". Las activaciones ReLU de las frutas de prueba lo confirman: las dos manzanas tienen las cuatro neuronas en cero (ninguna evidencia a favor de banano), mientras que los bananos activan las neuronas 2 y 3 con valores de 5.07 y 3.11.

Diferencia clave frente a NumPy. Keras usa Adam, un optimizador adaptativo que combina momento y una tasa de aprendizaje por parámetro, en lugar del descenso de gradiente simple. Por eso los valores numéricos de los pesos difieren de los de la Práctica 2 aunque la decisión sea la misma: ambos resuelven el mismo problema separable por trayectorias de optimización distintas.

Error observado en Colab	Causa	Solución aplicada
`NameError: name 'X_train' is not defined`	Se escaló o entrenó antes de crear la partición	Ejecutar `train_test_split` antes de `StandardScaler` y respetar el orden de las celdas
`NameError: name 'Dense' is not defined`	Se escribió `Dense(...)` sin importarlo	Usar `keras.layers.Dense(...)` o importarla explícitamente
`SyntaxError` en `or epoch ...`	Falta la letra `f` de `for`	Escribir `for epoch in range(epochs):` y revisar la sangría
La pérdida no baja de 0.69	Etiquetas con forma incorrecta o datos sin escalar	Verificar `y.reshape(-1,1)` y aplicar `StandardScaler`
Logs abundantes de TensorFlow	Nivel de log por defecto	`os.environ["TF_CPP_MIN_LOG_LEVEL"]="3"` antes de importar TensorFlow

En Colab una variable existe únicamente después de ejecutar la celda que la crea. Si el entorno se reinicia, conviene usar Entorno de ejecución → Ejecutar todo.

7. Práctica 4 · Clasificación multiclase con TensorFlow + Keras

Notebook: `04_Frutas_Multiclase_Keras.ipynb` · **Problema:** manzana (0), banano (1) y naranja (2) · **Modelo:** 3 → 6 → 3 con ReLU y salida **Softmax**, pérdida `sparse_categorical_crossentropy`, Adam con $\eta = 0.01$ y 300 épocas.

Se agrega una tercera categoría y el problema deja de ser binario. Este paso prepara el concepto de Softmax y de matriz de confusión antes de abordar MNIST, donde las clases serán diez. Las naranjas son más pesadas que las manzanas pero cortas y poco amarillas, de modo que no basta un umbral sobre una única característica: el modelo debe combinar las tres.

7.1 Dataset multiclase y preprocesamiento

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, ConfusionMatrixDisplay)
import random

random.seed(42); np.random.seed(42); tf.random.set_seed(42)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)
```

Salida de ejecución (evidencia)

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791644432.956849   25710 port.cc:153] oneDNN custom operations are on. You may see slightly
different numerical results due to floating-point round-off errors from different computation orders. To turn
them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791644434.533555   25710 port.cc:153] oneDNN custom operations are on. You may see slightly
different numerical results due to floating-point round-off errors from different computation orders. To turn
them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
TensorFlow: 2.21.0 | Keras: 3.15.1
```

```
X_multi = np.array([
    # MANZANAS - clase 0
    [150, 7.5, .10], [160, 8.0, .15], [145, 7.2, .05], [170, 8.3, .20], [155, 7.8, .10],
    [165, 8.1, .15], [140, 7.0, .05], [175, 8.5, .20], [158, 7.9, .10], [162, 8.2, .15],

    # BANANOS - clase 1
    [120, 18.0, .90], [125, 19.0, .95], [115, 17.5, .85], [130, 20.0, 1.00], [118, 18.5, .90],
```

```
    [123, 19.5, .95], [110, 17.0, .80], [128, 20.5, 1.00], [121, 18.8, .90], [126, 19.2, .95],

    # NARANJAS - clase 2
    [180, 8.0, .35], [190, 8.5, .40], [175, 7.8, .30], [200, 9.0, .45], [185, 8.2, .35],
    [195, 8.7, .40], [170, 7.5, .30], [205, 9.2, .45], [188, 8.4, .35], [192, 8.6, .40]
], dtype=float)

y_multi = np.array([0] * 10 + [1] * 10 + [2] * 10)
nombres_frutas = ["MANZANA", "BANANO", "NARANJA"]

print("X_multi:", X_multi.shape, "-> (30 frutas, 3 características)")
print("y_multi:", y_multi.shape, "-> clases:", np.unique(y_multi))
print("Ejemplos por clase:", np.bincount(y_multi))
print()
for c, n in enumerate(nombres_frutas):
    m = y_multi == c
    print(f"{n:<9} peso {X_multi[m,0].mean():6.1f} g | longitud {X_multi[m,1].mean():5.2f} cm | amarillo
{X_multi[m,2].mean():.2f}")
```

Salida de ejecución (evidencia)

```
X_multi: (30, 3) -> (30 frutas, 3 características)
y_multi: (30,) -> clases: [0 1 2]
Ejemplos por clase: [10 10 10]

MANZANA   peso  158.0 g | longitud  7.85 cm | amarillo 0.12
BANANO    peso  121.6 g | longitud 18.80 cm | amarillo 0.92
NARANJA   peso  188.0 g | longitud  8.39 cm | amarillo 0.38
```

```
# División estratificada 80/20 y escalado
X_train_multi, X_test_multi, y_train_multi, y_test_multi = train_test_split(
    X_multi, y_multi, test_size=0.2, random_state=42, stratify=y_multi
)

scaler_multi = StandardScaler()
X_train_multi = scaler_multi.fit_transform(X_train_multi)
X_test_multi = scaler_multi.transform(X_test_multi)

print("X_train_multi:", X_train_multi.shape, "| X_test_multi:", X_test_multi.shape)
print("Clases en entrenamiento:", np.bincount(y_train_multi))
print("Clases en prueba       :", np.bincount(y_test_multi))
print("Etiquetas de prueba    :", y_test_multi)
```

Salida de ejecución (evidencia)

```
X_train_multi: (24, 3) | X_test_multi: (6, 3)
Clases en entrenamiento: [8 8 8]
Clases en prueba       : [2 2 2]
Etiquetas de prueba    : [2 1 2 0 1 0]
```

7.2 Verificación numérica de la función Softmax

Softmax conserva el orden de los logits y los normaliza para que sumen 1. Con logits [2.0, 1.0, 0.1] produce [0.659, 0.2424, 0.0986] y `argmax` devuelve la posición 0.

```
# Verificación del comportamiento de Softmax
z_ejemplo = np.array([[2.0, 1.0, 0.1]])
exp = np.exp(z_ejemplo)
softmax_manual = exp / exp.sum()

print("Logits z            :", z_ejemplo[0])
print("Softmax manual      :", np.round(softmax_manual[0], 4), "| suma:", round(softmax_manual.sum(), 6))
print("Clase elegida argmax:", int(np.argmax(softmax_manual)))
print()
z_otro = np.array([[0.4, 2.8, 1.1]])
s_otro = np.exp(z_otro) / np.exp(z_otro).sum()
print("Logits z            :", z_otro[0])
```

```
print("Softmax             :", np.round(s_otro[0], 4), "| argmax:", int(np.argmax(s_otro)))
print("-> Softmax conserva el orden de los logits y los normaliza para que sumen 1.")
```

Salida de ejecución (evidencia)

```
Logits z            : [2.  1.  0.1]
Softmax manual      : [0.659  0.2424 0.0986] | suma: 1.0
Clase elegida argmax: 0

Logits z            : [0.4 2.8 1.1]
Softmax             : [0.0712 0.7853 0.1435] | argmax: 1
-> Softmax conserva el orden de los logits y los normaliza para que sumen 1.
```

7.3 Modelo, entrenamiento y curvas

```
tf.random.set_seed(42)

modelo_multi = keras.Sequential([
    keras.layers.Input(shape=(3,)),
    keras.layers.Dense(6, activation="relu", name="oculta"),
    keras.layers.Dense(3, activation="softmax", name="salida_softmax")
])

modelo_multi.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.01),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

modelo_multi.summary()
```

```
historial_multi = modelo_multi.fit(
    X_train_multi,
    y_train_multi,
    epochs=300,
    batch_size=4,
    validation_split=0.2,
    verbose=0
)

print("Error inicial:", round(historial_multi.history["loss"][0], 6))
print("Error final  :", round(historial_multi.history["loss"][-1], 6))
print()
print(f"{'Época':>7}{'Loss':>11}{'Acc':>8}{'Val loss':>11}{'Val acc':>9}")
for e in [0, 49, 99, 149, 199, 249, 299]:
    h = historial_multi.history
    print(f"{e+1:>7}{h['loss'][e]:>11.5f}{h['accuracy'][e]:>8.3f}"
          f"{h['val_loss'][e]:>11.5f}{h['val_accuracy'][e]:>9.3f}")
```

Salida de ejecución (evidencia)

```
Error inicial: 1.469859
Error final  : 0.005629

  Época       Loss     Acc   Val loss  Val acc
      1    1.46986   0.000    1.30948    0.000
     50    0.10030   0.947    0.00284    1.000
    100    0.05633   1.000    0.00054    1.000
    150    0.03043   1.000    0.00021    1.000
    200    0.01625   1.000    0.00011    1.000
    250    0.00920   1.000    0.00007    1.000
    300    0.00563   1.000    0.00005    1.000
```

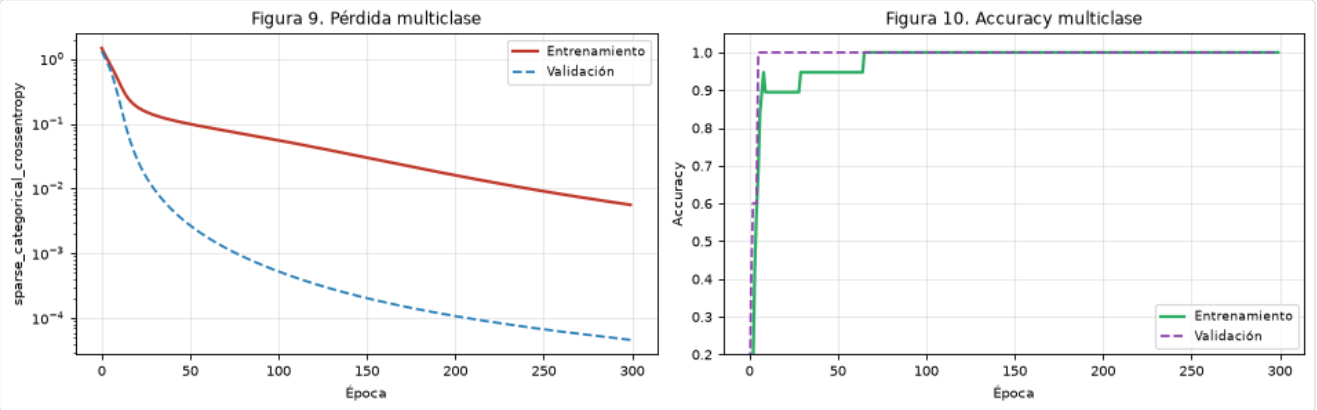


Figura 6. Pérdida y accuracy de entrenamiento y validación en la clasificación multiclase de frutas.

7.4 Evaluación y probabilidades por clase

7.5 Matriz de confusión y reporte de clasificación

El accuracy global no dice **qué** clases se confunden entre sí. En problemas multiclase la matriz de confusión es la herramienta adecuada y será la métrica principal en MNIST.

7.6 Frontera de decisión y confianza de la predicción

7.7 Predicción de una fruta nueva

7.8 Análisis de resultados de la Práctica 4

Indicador	Valor obtenido	Interpretación
Pérdida inicial	1.469859	Cerca de la referencia teórica −log(1/3) = 1.0986, propia de un modelo que aún no discrimina.
Pérdida final (época 300)	0.005629	Reducción del 99.6 %.
Accuracy de validación	1.0000 desde la época 50	Convergencia estable, sin sobreajuste apreciable.
Loss / Accuracy en prueba	0.003006 / 1.0000 (6 de 6)	Clasificación correcta de las seis frutas no vistas.
Matriz de confusión	diag(2, 2, 2) sin errores fuera de la diagonal	Ninguna clase se confunde con otra.
Probabilidades de la fruta nueva (190 g, 8.5 cm, 0.38)	P(naranja) = 1.0000 → NARANJA	Decisión correcta y con máxima confianza.
Confianza promedio en la malla de decisión	0.9360	Solo el 4.20 % del área presenta probabilidad máxima inferior a 0.60 (zonas de frontera difusa).

Cambios coordinados al pasar de binario a multiclase. Fueron necesarios tres ajustes simultáneos y coherentes entre sí: el número de neuronas de salida pasó de 1 a 3, la activación de salida pasó de Sigmoide a Softmax y la pérdida pasó de `binary_crossentropy` a `sparse_categorical_crossentropy`. Modificar solo uno de los tres produce errores de forma o entrenamientos que no convergen.

Por qué `sparse_categorical_crossentropy` y no `categorical_crossentropy`. Las etiquetas están codificadas como enteros (0, 1, 2). La variante sparse las acepta directamente; la variante sin sparse

exigiría vectores one-hot del tipo [1,0,0]. Elegir la combinación incorrecta es una fuente frecuente de fallos.

Lectura de la confianza. La probabilidad máxima permite identificar las zonas ambiguas del espacio de características. Ese indicador es el que, en un sistema real, justificaría derivar un caso a revisión humana en lugar de emitir una decisión automática.

8. Práctica 5 · MNIST multiclase con TensorFlow + Keras (sin redes convolucionales)

Notebook: `05_MNIST_Keras.ipynb` · **Problema:** reconocimiento de dígitos manuscritos, diez clases · **Modelo:** perceptrón multicapa **784 → 128 → 10** con ReLU y Softmax · **Entrenamiento:** 5 épocas, batch de 128, Adam con η = 0.001, validación del 10 %.

La misma lógica multiclase de la Práctica 4 se traslada a un problema estándar y real. No se genera ningún número: se clasifican imágenes en una de diez clases. **No se utiliza ninguna capa convolucional**, tal como exige la consigna, y el notebook lo verifica de forma programática.

8.1 Entorno y carga del dataset MNIST

```
import os
# Ambiente silencioso y determinista para TensorFlow (no altera los resultados)
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
os.environ["PYTHONHASHSEED"] = "0"

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import tensorflow as tf
from tensorflow import keras
import random, time, gc
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay

random.seed(42); np.random.seed(42); tf.random.set_seed(42)
print("TensorFlow:", tf.__version__, "| Keras:", keras.__version__)
print("Backend    :", keras.backend.backend())
print("Dispositivos:", [d.device_type for d in tf.config.list_physical_devices()])
```

Salida de ejecución (evidencia)

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791644462.543707   30570 port.cc:153] oneDNN custom operations are on. You may see slightly
different numerical results due to floating-point round-off errors from different computation orders. To turn
them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791644463.915123   30570 port.cc:153] oneDNN custom operations are on. You may see slightly
different numerical results due to floating-point round-off errors from different computation orders. To turn
them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
TensorFlow: 2.21.0 | Keras: 3.15.1
Backend    : tensorflow
Dispositivos: ['CPU']
```

```
ruta_mnist = keras.utils.get_file(
    "mnist.npz",
    "https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz",
)
```

```
with np.load(ruta_mnist, allow_pickle=False) as datos:
    x_train_mnist = datos["x_train"]                    # uint8 (60000, 28, 28)
    y_train_mnist = datos["y_train"]                    # uint8 (60000,)
    x_test_mnist = datos["x_test"].astype("float32") / 255.0   # normalizado [0,1]
    y_test_mnist = datos["y_test"]

print("Archivo:", ruta_mnist)
print()
print("Entrenamiento:", x_train_mnist.shape, y_train_mnist.shape, x_train_mnist.dtype)
print("Prueba       :", x_test_mnist.shape, y_test_mnist.shape, x_test_mnist.dtype)
print("Rango de píxeles (entrenamiento):", x_train_mnist.min(), "-", x_train_mnist.max())
print("Rango de píxeles (prueba)       :", x_test_mnist.min(), "-", x_test_mnist.max())
print("Clases:", np.unique(y_train_mnist))
print("Ejemplos por clase:", np.bincount(y_train_mnist))
print()
print("Primer ejemplo: etiqueta real =", y_train_mnist[0], "| forma =", x_train_mnist[0].shape)
print("Memoria del conjunto de entrenamiento:",
      round(x_train_mnist.nbytes / 1024**2, 1), "MB (uint8)")
```

Salida de ejecución (evidencia)

```
Archivo: /tmp/.keras/datasets/mnist.npz

Entrenamiento: (60000, 28, 28) (60000,) uint8
Prueba       : (10000, 28, 28) (10000,) float32
Rango de píxeles (entrenamiento): 0 - 255
Rango de píxeles (prueba)       : 0.0 - 1.0
Clases: [0 1 2 3 4 5 6 7 8 9]
Ejemplos por clase: [5923 6742 5958 6131 5842 5421 5918 6265 5851 5949]

Primer ejemplo: etiqueta real = 5 | forma = (28, 28)
Memoria del conjunto de entrenamiento: 44.9 MB (uint8)
```

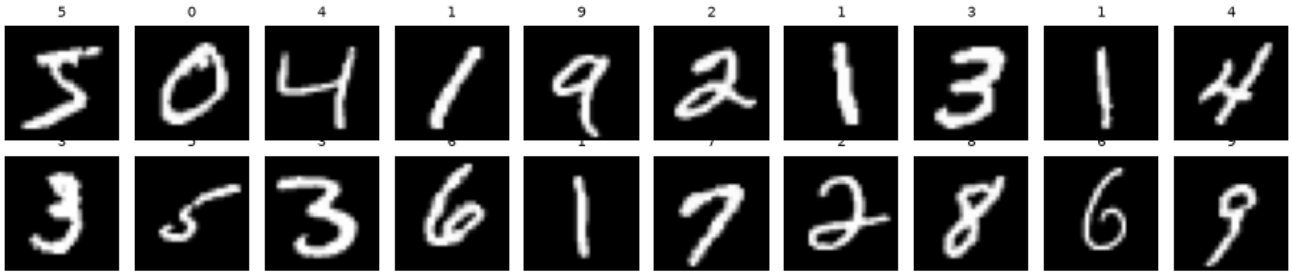


Figura 9. Primeras 20 imágenes de entrenamiento de MNIST con su etiqueta real.

8.2 Normalización y aplanado de la entrada

La red no «ve» el dígito como una persona: procesa valores numéricos de píxeles. Dividir entre 255 ubica todas las entradas en [0, 1], lo que estabiliza el descenso de gradiente. La operación `reshape(-1, 784)` equivale a la capa `Flatten`: convierte la matriz 28 × 28 en un vector de 784 características.

```
def normalizar_lote(imagenes):
    """Convierte un lote uint8 (n,28,28) a float32 (n,784) en [0,1]."""
    return imagenes.reshape(len(imagenes), -1).astype("float32") / 255.0


lote_demo = normalizar_lote(x_train_mnist[:5])
print("Lote de 5 imágenes ->", lote_demo.shape, lote_demo.dtype)
print("Mínimo:", lote_demo.min(), "| Máximo:", lote_demo.max())
print()
print("Primeros 12 valores del vector de 784 píxeles (zona de borde, casi todo ceros):")
print(np.round(lote_demo[0][:12], 3))
print()
zona = lote_demo[0].reshape(28, 28)[10:18, 8:20]
print("Bloque central 8x12 de la misma imagen (donde sí hay trazo):")
print(np.round(zona, 2))
```

```
print()
print("Verificación: 784 =", 28 * 28, "->", lote_demo.shape[1] == 28 * 28)
```

Salida de ejecución (evidencia)

```
Lote de 5 imágenes -> (5, 784) float32
Mínimo: 0.0 | Máximo: 1.0

Primeros 12 valores del vector de 784 píxeles (zona de borde, casi todo ceros):
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

Bloque central 8x12 de la misma imagen (donde sí hay trazo):
[[0.   0.05 0.   0.6  0.99 0.35 0.   0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.55 0.99 0.75 0.01 0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.04 0.75 0.99 0.27 0.   0.   0.   0.   0.  ]
 [0.   0.   0.   0.   0.14 0.95 0.88 0.63 0.42 0.   0.   0.  ]
 [0.   0.   0.   0.   0.   0.32 0.94 0.99 0.99 0.47 0.1  0.  ]
 [0.   0.   0.   0.   0.   0.   0.18 0.73 0.99 0.99 0.59 0.11]
 [0.   0.   0.   0.   0.   0.   0.   0.06 0.36 0.99 0.99 0.73]
 [0.   0.   0.   0.   0.   0.   0.   0.   0.   0.98 0.99 0.98]]

Verificación: 784 = 784 -> True
```

8.3 Arquitectura y verificación de la ausencia de convoluciones

```
tf.random.set_seed(42)

modelo_mnist = keras.Sequential([
    keras.layers.Input(shape=(784,)),
    keras.layers.Dense(128, activation="relu", name="oculta_128"),
    keras.layers.Dense(10, activation="softmax", name="salida_10")
])

modelo_mnist.summary()

tipos = [capa.__class__.__name__ for capa in modelo_mnist.layers]
print()
print("Capas del modelo            :", tipos)
print("¿Hay alguna capa convolucional?", any("Conv" in t for t in tipos))
print("¿Hay alguna capa de pooling?  ", any("Pooling" in t for t in tipos))
print("Parámetros entrenables      :", f"{modelo_mnist.count_params():,}")
print("-> Verificación de la restricción: red 100 % densa (perceptrón multicapa).")
```

Salida de ejecución (evidencia)

```
Capas del modelo            : ['Dense', 'Dense']
¿Hay alguna capa convolucional? False
¿Hay alguna capa de pooling?   False
Parámetros entrenables      : 101,770
-> Verificación de la restricción: red 100 % densa (perceptrón multicapa).
```

8.4 Compilación manual: pérdida, optimizador y paso de entrenamiento

En lugar de `compile()` + `fit()` se declaran explícitamente la función de pérdida y el optimizador, y se escribe el paso de entrenamiento con `tf.GradientTape`. Es el mismo ciclo que Keras ejecuta internamente y el mismo que se escribirá en PyTorch en la Práctica 6. El motivo técnico de esta decisión se explica en la sección 3 de este documento.

```
perdida_fn = keras.losses.SparseCategoricalCrossentropy()
optimizador = keras.optimizers.Adam(learning_rate=0.001)
metrica_acc = keras.metrics.SparseCategoricalAccuracy()

print("Pérdida    :", perdida_fn.__class__.__name__)
print("Optimizador:", optimizador.__class__.__name__, "| lr =", optimizador.learning_rate)


@tf.function
def paso_entrenamiento(xb, yb):
```

```
    """Un paso completo: forward -> loss -> backpropagation -> actualización."""
    with tf.GradientTape() as cinta:
        logits = modelo_mnist(xb, training=True)
        perdida = perdida_fn(yb, logits)
    gradientes = cinta.gradient(perdida, modelo_mnist.trainable_variables)   # backpropagation
    optimizador.apply_gradients(zip(gradientes, modelo_mnist.trainable_variables))  # actualización
    return perdida


def evaluar_conjunto(x_datos, y_datos, tam_lote=1000, normalizar=False):
    """Evalúa pérdida y accuracy por lotes, sin materializar todas las predicciones."""
    perdidas, aciertos, total = 0.0, 0, 0
    clases, confianzas = [], []
    for inicio in range(0, len(x_datos), tam_lote):
        xb = x_datos[inicio:inicio + tam_lote]
        yb = y_datos[inicio:inicio + tam_lote]
        if normalizar:
            xb = normalizar_lote(xb)
        else:
            xb = xb.reshape(len(xb), -1)
        probs = modelo_mnist(xb, training=False).numpy()
        perdidas += float(perdida_fn(yb, probs)) * len(yb)
        aciertos += int((probs.argmax(axis=1) == yb).sum())
        total += len(yb)
        clases.append(probs.argmax(axis=1))
        confianzas.append(probs.max(axis=1))
    return (perdidas / total, aciertos / total,
            np.concatenate(clases), np.concatenate(confianzas))


print()
print("Funciones de entrenamiento y evaluación definidas correctamente")
```

Salida de ejecución (evidencia)

```
Pérdida    : SparseCategoricalCrossentropy
Optimizador: Adam | lr = <Variable path=adam/learning_rate, shape=(), dtype=float32,
value=0.0010000000474974513>

Funciones de entrenamiento y evaluación definidas correctamente
```

8.5 Estado inicial del modelo sin entrenar

```
xb0 = normalizar_lote(x_train_mnist[:1])
yb0 = y_train_mnist[:1]
probs0 = modelo_mnist(xb0, training=False).numpy()

print("Etiqueta real de la imagen 0:", yb0[0])
print("Probabilidades sin entrenar :", np.round(probs0[0], 4))
print("Suma de probabilidades    :", round(float(probs0[0].sum()), 6))
print("Pérdida inicial             :", round(float(perdida_fn(yb0, probs0)), 4))
print("Referencia teórica -ln(1/10):", round(-np.log(0.1), 4))
```

Salida de ejecución (evidencia)

```
Etiqueta real de la imagen 0: 5
Probabilidades sin entrenar : [0.0849 0.0669 0.0561 0.13   0.0804 0.1989 0.1206 0.1231 0.1115 0.0275]
Suma de probabilidades    : 1.0
Pérdida inicial             : 1.615
Referencia teórica -ln(1/10): 2.3026
```

8.6 Entrenamiento (5 épocas) y curvas de aprendizaje

```
BATCH_SIZE = 128
EPOCHS = 5
N_VALIDACION = 6000
n_entrenamiento = len(x_train_mnist) - N_VALIDACION

indices_train = np.arange(n_entrenamiento)
```

```
indices_val = np.arange(n_entrenamiento, len(x_train_mnist))

historial = {"loss": [], "accuracy": [], "val_loss": [], "val_accuracy": []}
t0 = time.time()

for epoch in range(EPOCHS):
    # ---------- ENTRENAMIENTO ----------
    np.random.shuffle(indices_train)
    perdidas_epoca, aciertos_epoca, vistos = 0.0, 0, 0

    for inicio in range(0, n_entrenamiento, BATCH_SIZE):
        idx = indices_train[inicio:inicio + BATCH_SIZE]
        xb = normalizar_lote(x_train_mnist[idx])
        yb = y_train_mnist[idx]

        perdida = paso_entrenamiento(xb, yb)          # forward + loss + backward + update

        probs = modelo_mnist(xb, training=False).numpy()
        perdidas_epoca += float(perdida) * len(idx)
        aciertos_epoca += int((probs.argmax(axis=1) == yb).sum())
        vistos += len(idx)

    # ---------- VALIDACIÓN ----------
    perd_val, aciertos_val, total_val = 0.0, 0, 0
    for inicio in range(0, N_VALIDACION, 512):
        idx = indices_val[inicio:inicio + 512]
        xb = normalizar_lote(x_train_mnist[idx])
        yb = y_train_mnist[idx]
        probs = modelo_mnist(xb, training=False).numpy()
        perd_val += float(perdida_fn(yb, probs)) * len(yb)
        aciertos_val += int((probs.argmax(axis=1) == yb).sum())
        total_val += len(yb)

    historial["loss"].append(perdidas_epoca / vistos)
    historial["accuracy"].append(aciertos_epoca / vistos)
    historial["val_loss"].append(perd_val / total_val)
    historial["val_accuracy"].append(aciertos_val / total_val)
    gc.collect()

tiempo = time.time() - t0
print(f"{'Época':>6}{'Loss':>10}{'Accuracy':>10}{'Val loss':>11}{'Val acc':>10}")
for i in range(EPOCHS):
    print(f"{i+1:>6}{historial['loss'][i]:>10.4f}{historial['accuracy'][i]:>10.4f}"
          f"{historial['val_loss'][i]:>11.4f}{historial['val_accuracy'][i]:>10.4f}")
print()
print(f"Tiempo total: {tiempo:.1f} s ({tiempo/EPOCHS:.1f} s por época)")
print(f"Lotes por época: {int(np.ceil(n_entrenamiento / BATCH_SIZE))}")
```

Salida de ejecución (evidencia)

```
 Época      Loss  Accuracy   Val loss   Val acc
     1    0.3740    0.9013     0.1691    0.9553
     2    0.1700    0.9539     0.1283    0.9658
     3    0.1217    0.9673     0.1023    0.9723
     4    0.0946    0.9747     0.0922    0.9740
     5    0.0760    0.9805     0.0882    0.9760

Tiempo total: 8.5 s (1.7 s por época)
Lotes por época: 422
```

```
fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.6))
epocas = range(1, EPOCHS + 1)

axes[0].plot(epocas, historial["loss"], marker="o", label="Entrenamiento", color="#c0392b", lw=2)
axes[0].plot(epocas, historial["val_loss"], marker="s", label="Validación", color="#2980b9", lw=2,
ls="--")
axes[0].set_title("Figura 15. Error durante el entrenamiento")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("Loss"); axes[0].legend(fontsize=8)

axes[1].plot(epocas, historial["accuracy"], marker="o", label="Entrenamiento", color="#27ae60", lw=2)
axes[1].plot(epocas, historial["val_accuracy"], marker="s", label="Validación", color="#8e44ad", lw=2,
ls="--")
```

```
axes[1].set_ylim(0.88, 1.0)
axes[1].set_title("Figura 16. Precisión durante el entrenamiento")
axes[1].set_xlabel("Época"); axes[1].set_ylabel("Accuracy"); axes[1].legend(fontsize=8)

plt.tight_layout(); plt.savefig("fig10_mnist_curvas_keras.png"); plt.show()
plt.close(fig)
_ = gc.collect()
```

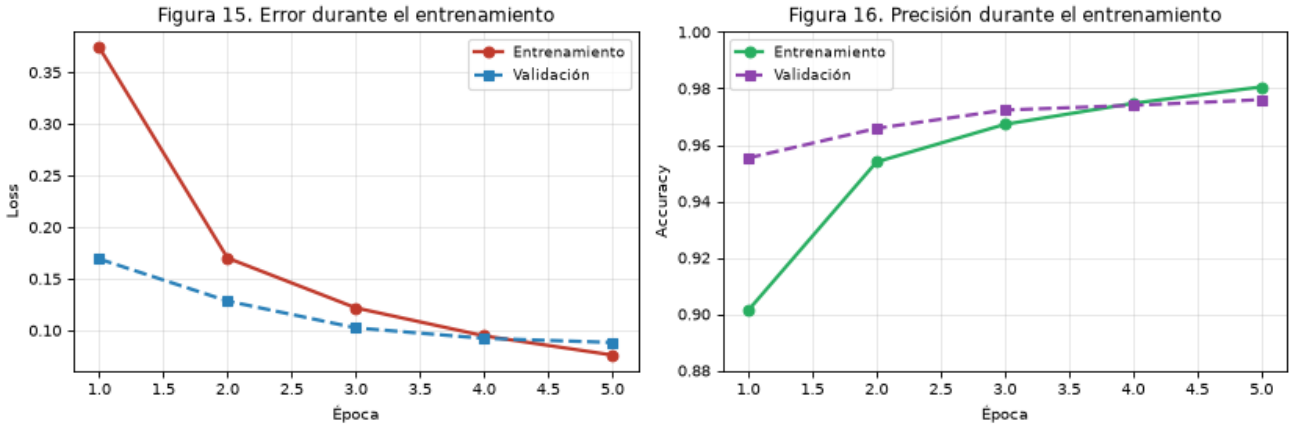


Figura 10. Curvas de pérdida y precisión (entrenamiento y validación) de MNIST con Keras.

8.7 Evaluación final sobre las 10 000 imágenes de prueba

```
test_loss, test_accuracy, clases_test, confianzas_test = evaluar_conjunto(
    x_test_mnist, y_test_mnist, tam_lote=1000
)

print("=== RESULTADOS MNIST · TensorFlow + Keras ===")
print("Error de prueba    :", round(test_loss, 6))
print("Accuracy de prueba :", round(test_accuracy, 6))
print("Imágenes correctas :", int(round(test_accuracy * len(y_test_mnist))), "de", len(y_test_mnist))
print("Imágenes erróneas  :", len(y_test_mnist) - int(round(test_accuracy * len(y_test_mnist))))
print()
print("Confianza promedio en aciertos:",
      round(float(confianzas_test[clases_test == y_test_mnist].mean()), 4))
print("Confianza promedio en errores :",
      round(float(confianzas_test[clases_test != y_test_mnist].mean()), 4))
print("-> El modelo está mucho menos seguro cuando se equivoca: la probabilidad máxima es")
print("   una señal útil para decidir cuándo requerir revisión humana.")
```

Salida de ejecución (evidencia)

```
=== RESULTADOS MNIST · TensorFlow + Keras ===
Error de prueba    : 0.089279
Accuracy de prueba : 0.9735
Imágenes correctas : 9735 de 10000
Imágenes erróneas  : 265

Confianza promedio en aciertos: 0.9764
Confianza promedio en errores : 0.6941
-> El modelo está mucho menos seguro cuando se equivoca: la probabilidad máxima es
   una señal útil para decidir cuándo requerir revisión humana.
```

```
probs10 = modelo_mnist(x_test_mnist[:10].reshape(10, 784),
                                    training=False).numpy()

print("=== PRIMERAS 10 IMÁGENES DE PRUEBA ===")
print(f"{'Imagen':>7}{'Real':>6}{'Predicción':>12}{'Confianza':>11}{'Resultado':>11}")
for i in range(10):
    prediccion = int(np.argmax(probs10[i]))
    conf = float(probs10[i].max())
    ok = "ACIERTO" if prediccion == y_test_mnist[i] else "ERROR"
    print(f"{i+1:>7}{int(y_test_mnist[i]):>6}{prediccion:>12}{conf:>11.4f}{ok:>11}")
```

```
print()
print("Vector completo de probabilidades de la imagen 1:")
print(np.round(probs10[0], 6), "-> suma:", round(float(probs10[0].sum()), 6))
```

Salida de ejecución (evidencia)

```
=== PRIMERAS 10 IMÁGENES DE PRUEBA ===
 Imagen  Real  Predicción  Confianza  Resultado
      1     7           7     0.9997    ACIERTO
      2     2           2     0.9994    ACIERTO
      3     1           1     0.9953    ACIERTO
      4     0           0     0.9999    ACIERTO
      5     4           4     0.9899    ACIERTO
      6     1           1     0.9934    ACIERTO
      7     4           4     0.9980    ACIERTO
      8     9           9     0.9862    ACIERTO
      9     5           6     0.7301      ERROR
     10     9           9     0.9741    ACIERTO

Vector completo de probabilidades de la imagen 1:
[0.00000e+00 0.00000e+00 3.10000e-05 2.91000e-04 0.00000e+00 3.00000e-06
 0.00000e+00 9.99653e-01 7.00000e-06 1.30000e-05] -> suma: 1.0
```

```
fig, axes = plt.subplots(2, 5, figsize=(11.5, 3.0))
for i, ax in enumerate(axes.ravel()):
    pred = int(np.argmax(probs10[i]))
    real = int(y_test_mnist[i])
    ax.imshow(x_test_mnist[i].reshape(28, 28), cmap="gray")
    ax.set_title(f"Real: {real} · Pred: {pred}", fontsize=9,
                 color="#27ae60" if pred == real else "#c0392b")
    ax.axis("off")
fig.suptitle("Figura 17. Predicciones sobre las primeras 10 imágenes de prueba", y=1.06)
plt.tight_layout(); plt.savefig("fig11_mnist_predicciones.png"); plt.show()
plt.close(fig)
_ = gc.collect()
```

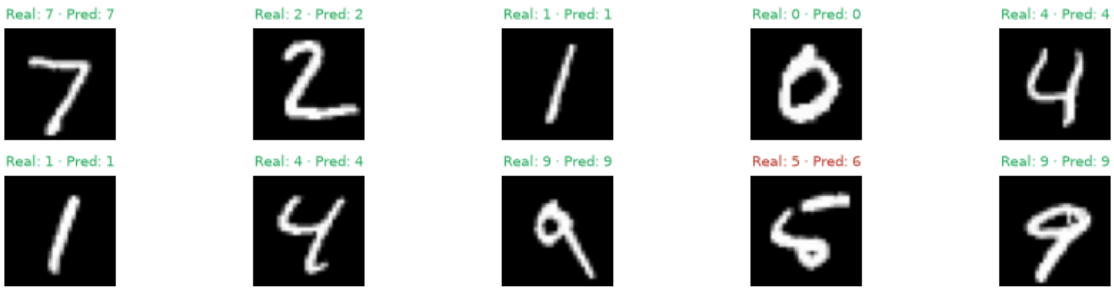


Figura 11. Predicciones sobre las primeras 10 imágenes del conjunto de prueba.

8.8 Análisis de errores del modelo

Un accuracy alto no debe convertirse en una afirmación de perfección. Analizar los errores revela los patrones de confusión del clasificador.

```
errores = np.where(clases_test != y_test_mnist)[0]

print("Cantidad total de errores:", len(errores), "de", len(y_test_mnist))
print("Tasa de error            :", f"{len(errores)/len(y_test_mnist)*100:.2f} %")
print()
print("Primeros 15 errores (índice | real -> predicho | confianza):")
for idx in errores[:15]:
    print(f"  {idx:>5} | {y_test_mnist[idx]} -> {clases_test[idx]} | {confianzas_test[idx]:.4f}")
```

Salida de ejecución (evidencia)

```
Cantidad total de errores: 265 de 10000
Tasa de error            : 2.65 %

Primeros 15 errores (índice | real -> predicho | confianza):
      8 | 5 -> 6 | 0.7301
    247 | 4 -> 2 | 0.5578
    259 | 6 -> 0 | 0.8357
    321 | 2 -> 7 | 0.9853
    340 | 5 -> 3 | 0.8896
    445 | 6 -> 0 | 0.8496
    478 | 5 -> 8 | 0.5151
    582 | 8 -> 2 | 0.6241
    591 | 8 -> 3 | 0.5062
    619 | 1 -> 8 | 0.5589
    628 | 3 -> 9 | 0.8086
    659 | 2 -> 8 | 0.5485
    684 | 7 -> 3 | 0.8381
    691 | 8 -> 4 | 0.9721
    707 | 4 -> 9 | 0.8640
```

```
cm = confusion_matrix(y_test_mnist, clases_test)
cm_norm = cm.astype("float") / cm.sum(axis=1, keepdims=True)

fig, ax = plt.subplots(figsize=(5.6, 4.6))
ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(
    ax=ax, cmap="Blues", colorbar=True, values_format="d", xticks_rotation=0)
ax.set_title("Figura 18. Matriz de confusión - MNIST (10 000 imágenes)")
plt.tight_layout(); plt.savefig("fig12_mnist_confusion.png"); plt.show()
plt.close(fig)
_ = gc.collect()

fig, ax = plt.subplots(figsize=(5.6, 4.6))
ConfusionMatrixDisplay(cm_norm, display_labels=range(10)).plot(
    ax=ax, cmap="Blues", colorbar=True, values_format=".3f", xticks_rotation=0)
ax.set_title("Figura 19. Matriz normalizada por clase real (recall)")
plt.tight_layout(); plt.savefig("fig12b_mnist_confusion_norm.png"); plt.show()
plt.close(fig)
_ = gc.collect()
```

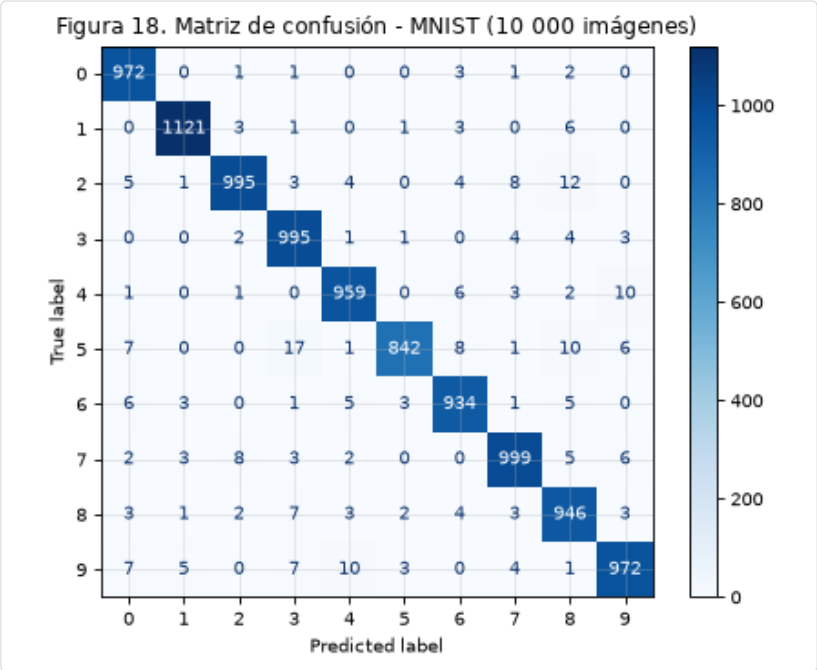


Figura 12. Matriz de confusión absoluta de MNIST con Keras (10 000 imágenes de prueba).

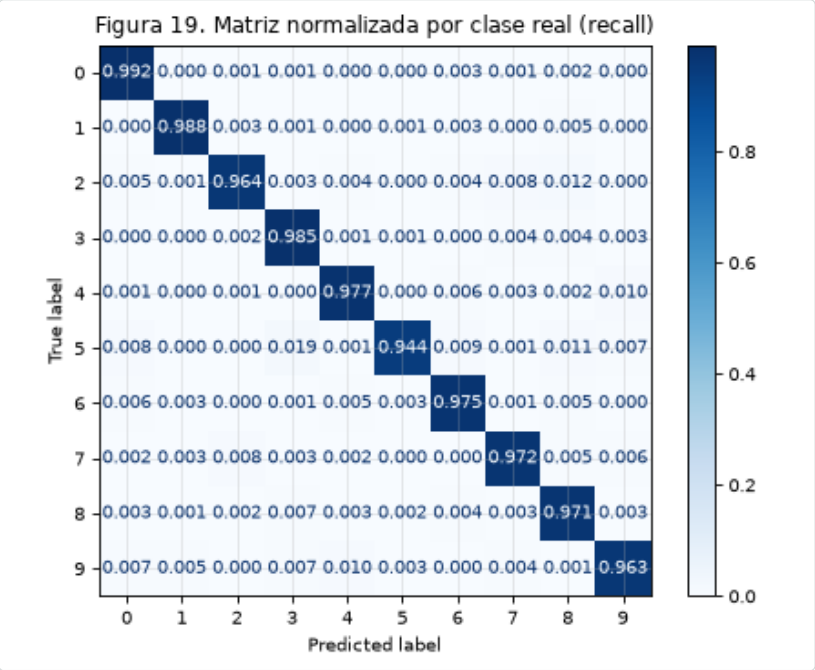


Figura 13. Matriz de confusión normalizada por clase real (recall por dígito).

```
print(classification_report(y_test_mnist, clases_test,
                           target_names=[str(d) for d in range(10)], digits=4))

pares = sorted([(cm[r, c], r, c) for r in range(10) for c in range(10) if r != c], reverse=True)
print("Top 5 confusiones (real -> predicho):")
for cant, r, c in pares[:5]:
    print(f"   {r} -> {c} : {cant} casos ({cant/cm[r].sum()*100:.2f} % de los dígitos '{r}')")
print()
print("Clase con mejor recall :", int(np.argmax(np.diag(cm_norm))),
      f"({np.diag(cm_norm).max():.4f})")
print("Clase con peor  recall :", int(np.argmin(np.diag(cm_norm))),
      f"({np.diag(cm_norm).min():.4f})")
```

Salida de ejecución (evidencia)

```
              precision    recall  f1-score   support

           0     0.9691    0.9918    0.9803       980
           1     0.9885    0.9877    0.9881      1135
           2     0.9832    0.9641    0.9736      1032
           3     0.9614    0.9851    0.9731      1010
           4     0.9736    0.9766    0.9751       982
           5     0.9883    0.9439    0.9656       892
           6     0.9709    0.9749    0.9729       958
           7     0.9756    0.9718    0.9737      1028
           8     0.9527    0.9713    0.9619       974
           9     0.9720    0.9633    0.9676      1009

    accuracy                         0.9735     10000
   macro avg     0.9735    0.9731    0.9732     10000
weighted avg     0.9737    0.9735    0.9735     10000

Top 5 confusiones (real -> predicho):
   5 -> 3 : 17 casos (1.91 % de los dígitos '5')
   2 -> 8 : 12 casos (1.16 % de los dígitos '2')
   9 -> 4 : 10 casos (0.99 % de los dígitos '9')
   5 -> 8 : 10 casos (1.12 % de los dígitos '5')
   4 -> 9 : 10 casos (1.02 % de los dígitos '4')

Clase con mejor recall : 0 (0.9918)
Clase con peor  recall : 5 (0.9439)
```

```
n_mostrar = 9
fig, axes = plt.subplots(1, n_mostrar, figsize=(11.5, 2.2))
```

```
for ax, idx in zip(axes, errores[:n_mostrar]):
    ax.imshow(x_test_mnist[idx].reshape(28, 28), cmap="gray")
    ax.set_title(f"Real {y_test_mnist[idx]}\nPred {clases_test[idx]}", fontsize=8, color="#c0392b")
    ax.axis("off")
fig.suptitle("Figura 20. Ejemplos de dígitos clasificados incorrectamente", y=1.14)
plt.tight_layout(); plt.savefig("fig13_mnist_errores.png"); plt.show()
plt.close(fig)
_ = gc.collect()
```

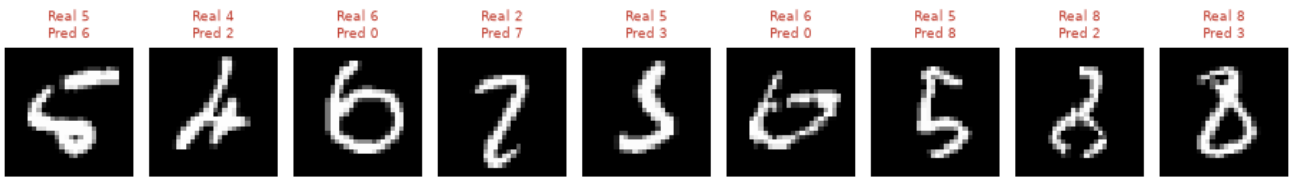


Figura 14. Ejemplos de dígitos clasificados incorrectamente por el modelo de Keras.

8.9 Experimento complementario: capacidad de la capa oculta

Se entrenan tres variantes (32, 128 y 256 neuronas) durante 2 épocas sobre un subconjunto estratificado de 18 000 imágenes, con la misma semilla y los mismos datos, para observar la relación entre capacidad, precisión y costo computacional.

```
def entrenar_variante(n_ocultas, x_idx, epochs=2, lr=0.001, semilla=42):
    """Entrena una red 784 -> n_ocultas -> 10 y devuelve métricas y tiempo."""
    tf.random.set_seed(semilla)
    m = keras.Sequential([
        keras.layers.Input(shape=(784,)),
        keras.layers.Dense(n_ocultas, activation="relu"),
        keras.layers.Dense(10, activation="softmax")
    ])
    opt = keras.optimizers.Adam(learning_rate=lr)
    loss_fn = keras.losses.SparseCategoricalCrossentropy()

    @tf.function
    def paso(xb, yb):
        with tf.GradientTape() as cinta:
            perdida = loss_fn(yb, m(xb, training=True))
        g = cinta.gradient(perdida, m.trainable_variables)
        opt.apply_gradients(zip(g, m.trainable_variables))
        return perdida

    idx = np.array(x_idx)
    np.random.shuffle(idx)
    t0 = time.time()
    for _ in range(epochs):
        for i in range(0, len(idx), BATCH_SIZE):
            sel = idx[i:i + BATCH_SIZE]
            paso(normalizar_lote(x_train_mnist[sel]), y_train_mnist[sel])
    dt = time.time() - t0

    loss_t, acc_t, _, _ = evaluar_variante(m, loss_fn)
    params = m.count_params()
    del m
    keras.backend.clear_session()
    gc.collect()
    return params, loss_t, acc_t, dt


def evaluar_variante(m, loss_fn, tam_lote=1000):
    perdidas, aciertos, total = 0.0, 0, 0
    for inicio in range(0, len(x_test_mnist), tam_lote):
        xb = x_test_mnist[inicio:inicio + tam_lote].reshape(tam_lote, -1)
        yb = y_test_mnist[inicio:inicio + tam_lote]
        probs = m(xb, training=False).numpy()
        perdidas += float(loss_fn(yb, probs)) * len(yb)
```

```
        aciertos += int((probs.argmax(axis=1) == yb).sum())
        total += len(yb)
    return perdidas / total, aciertos / total, None, None


idx_sub = np.arange(0, n_entrenamiento, 3)     # 18 000 imágenes estratificadas por posición
print("Subconjunto del experimento:", len(idx_sub), "imágenes")
print()
print(f"{'Neuronas':>9}{'Parámetros':>13}{'Test loss':>11}{'Test acc':>10}{'Segundos':>10}")
print("-" * 53)
resultados_cap = []
for n_ocultas in [32, 128, 256]:
    params, loss_t, acc_t, dt = entrenar_variante(n_ocultas, idx_sub)
    resultados_cap.append((n_ocultas, params, loss_t, acc_t, dt))
    print(f"{n_ocultas:>9}{params:>13,}{loss_t:>11.4f}{acc_t:>10.4f}{dt:>10.1f}")
```

Salida de ejecución (evidencia)

```
Subconjunto del experimento: 18000 imágenes

 Neuronas   Parámetros  Test loss  Test acc  Segundos
-----------------------------------------------------
       32       25,450     0.3170    0.9124       0.5
      128      101,770     0.2500    0.9311       0.7
      256      203,530     0.2247    0.9328       0.8
```

```
fig, ax = plt.subplots(figsize=(6.2, 3.8))
ax.plot([r[0] for r in resultados_cap], [r[3] for r in resultados_cap],
        marker="o", lw=2, color="#16a085")
for n, _, _, at, _ in resultados_cap:
    ax.annotate(f"{at:.4f}", (n, at), textcoords="offset points",
                xytext=(0, 9), ha="center", fontsize=9)
ax.set_xticks([32, 128, 256])
ax.set_xlabel("Neuronas en la capa oculta")
ax.set_ylabel("Accuracy en prueba")
ax.set_ylim(min(r[3] for r in resultados_cap) - 0.01, 1.0)
ax.set_title("Figura 21. Accuracy vs capacidad de la capa oculta (2 épocas, 18 000 imágenes)")
plt.tight_layout(); plt.savefig("fig14_mnist_capacidad.png"); plt.show()
plt.close(fig)
_ = gc.collect()
```

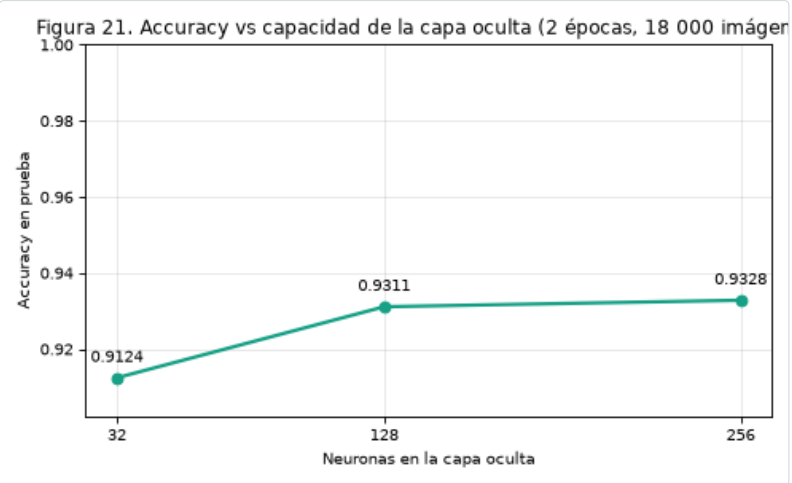


Figura 15. Accuracy en función del número de neuronas de la capa oculta (2 épocas, 18 000 imágenes).

8.10 Análisis de resultados de la Práctica 5

Indicador	Valor obtenido	Interpretación
Arquitectura	784 → 128 → 10 (ReLU + Softmax)	Sin capas convolucionales ni de pooling (verificado por código).
Parámetros entrenables	101 770	100 480 en la capa oculta y 1 290 en la salida.
Pérdida de entrenamiento (época 1 → 5)	0.3740 → 0.0760	Descenso sostenido.
Accuracy de entrenamiento (época 5)	0.9805	—
Pérdida / Accuracy de validación (época 5)	0.0882 / 0.9760	Acompaña a la de entrenamiento sin divergencia: no hay sobreajuste apreciable en 5 épocas.
Pérdida en prueba	**0.089279**	Sobre las 10 000 imágenes no vistas.
Accuracy en prueba	**0.9735**	9 735 aciertos y 265 errores (tasa de error 2.65 %).
Confianza promedio en aciertos	0.9764	El modelo está muy seguro cuando acierta.
Confianza promedio en errores	0.6941	Y notablemente menos seguro cuando falla: señal útil para rechazos.
Mejor / peor recall por clase	Dígito 0 (0.9918) / dígito 5 (0.9439)	—
Confusiones más frecuentes	5→3 (17), 2→8 (12), 9→4 (10), 5→8 (10), 4→9 (10)	Pares morfológicamente similares, coherente con un modelo que solo ve píxeles aplanados.
Tiempo de entrenamiento	8.7 s (1.7 s por época, CPU de 2 núcleos)	422 lotes por época.

Experimento de capacidad. Con 2 épocas y 18 000 imágenes: 32 neuronas alcanzan 0.9124 de accuracy (25 450 parámetros, 0.5 s); 128 neuronas llegan a 0.9311 (101 770 parámetros, 0.7 s) y 256 neuronas a 0.9328 (203 530 parámetros, 0.8 s). Duplicar la capacidad de 128 a 256 neuronas duplica también los parámetros y el tiempo, pero mejora el accuracy en apenas 0.17 puntos porcentuales: **más parámetros no siempre significan mejor modelo**. La diferencia relevante está entre 32 y 128 neuronas.

Por qué una red densa alcanza el 97.35 % sin convoluciones. Los dígitos MNIST están centrados y normalizados, de modo que un perceptrón multicapa puede aprender combinaciones lineales de píxeles suficientemente discriminativas. El costo de prescindir de convoluciones es doble: el modelo es sensible a la posición del dígito y concentra el 98.7 % de sus parámetros en la primera capa, frente a los pocos cientos que requeriría un filtro convolucional.

Lectura de la pérdida inicial. Antes de entrenar, la pérdida del primer ejemplo es 1.615 y las probabilidades son cercanas a 0.1 por dígito. La referencia teórica para diez clases equiprobables es $-\log(0.1) = 2.3026$; el valor observado es menor porque la inicialización aleatoria ya introduce un ligero sesgo, no porque el modelo «sepa» algo.

9. Práctica 6 · MNIST multiclase con PyTorch

Notebook: `06_MNIST_PyTorch.ipynb` · **Problema:** el mismo de la Práctica 5 · **Modelo:** clase `RedMNIST(nn.Module)` con arquitectura 784 → 128 → 10 · **Entrenamiento:** 3 épocas, batch de 64, Adam con $\eta = 0.001$.

La arquitectura se mantiene esencialmente igual; la diferencia es que el ciclo de entrenamiento aparece de manera **explícita**, lo que hace visible cada paso del aprendizaje: limpiar gradientes, propagar hacia adelante, calcular la pérdida, propagar hacia atrás y actualizar los pesos.

9.1 Entorno, dataset y DataLoaders

`transforms.ToTensor()` convierte cada imagen a un tensor de forma (1, 28, 28) y la escala automáticamente a [0, 1]: hace el mismo trabajo que la división manual entre 255 de la Práctica 5. Un **batch** es un pequeño grupo de ejemplos procesado antes de una actualización; `shuffle=True` se aplica solo en entrenamiento.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update({
    "figure.dpi": 80,
    "savefig.dpi": 80,
    "savefig.bbox": None,
    "figure.autolayout": False,
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 9,
})
plt.rcParams["font.family"] = "DejaVu Sans"

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import time
from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay

torch.manual_seed(42)
print("PyTorch    :", torch.__version__)
print("CUDA disponible:", torch.cuda.is_available())
print("Dispositivo usado:", "cuda" if torch.cuda.is_available() else "cpu")
```

Salida de ejecución (evidencia)

```
PyTorch    : 2.14.1+cpu
CUDA disponible: False
Dispositivo usado: cpu
```

```
transform = transforms.ToTensor()      # PIL/numpy -> tensor (1,28,28) en [0,1]

# La primera ejecución descarga MNIST (~11 MB) a la carpeta ./data.
# Se silencia la barra de progreso de la descarga para que la evidencia del
# notebook quede limpia; en Colab la descarga se muestra de forma nativa.
import contextlib
import urllib.request
import torchvision.datasets.utils as tvutils


def _descarga_sin_barra(url, filename, chunk_size=1024 * 256):
```

```
    """Descarga equivalente a la de torchvision, pero sin barra de progreso."""
    with urllib.request.urlopen(url) as respuesta, open(filename, "wb") as archivo:
        while True:
            bloque = respuesta.read(chunk_size)
            if not bloque:
                break
            archivo.write(bloque)


@contextlib.contextmanager
def descarga_silenciosa():
    original = tvutils._urlretrieve
    tvutils._urlretrieve = _descarga_sin_barra
    try:
        yield
    finally:
        tvutils._urlretrieve = original


with descarga_silenciosa():
    train_dataset = datasets.MNIST(root="./data", train=True, download=True,
                                   transform=transform)
    test_dataset = datasets.MNIST(root="./data", train=False, download=True,
                                   transform=transform)


print("Imágenes de entrenamiento:", len(train_dataset))
print("Imágenes de prueba       :", len(test_dataset))

imagen0, etiqueta0 = train_dataset[0]
print()
print("Tensor de la primera imagen:", tuple(imagen0.shape), imagen0.dtype)
print("Rango de píxeles           :", float(imagen0.min()), "-", float(imagen0.max()))
print("Etiqueta real              :", etiqueta0)
```

Salida de ejecución (evidencia)

```
Imágenes de entrenamiento: 60000
Imágenes de prueba       : 10000

Tensor de la primera imagen: (1, 28, 28) torch.float32
Rango de píxeles           : 0.0 - 1.0
Etiqueta real              : 5
```

```
BATCH_SIZE = 64

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print("Lotes de entrenamiento:", len(train_loader),
      f"({len(train_dataset)} imágenes / {BATCH_SIZE})")
print("Lotes de prueba       :", len(test_loader))

imagenes_lote, etiquetas_lote = next(iter(train_loader))
print()
print("Forma de un lote de imágenes:", tuple(imagenes_lote.shape), "-> (lote, canales, alto, ancho)")
print("Forma de un lote de etiquetas:", tuple(etiquetas_lote.shape))
print("Etiquetas del primer lote   :", etiquetas_lote[:16].tolist())
```

Salida de ejecución (evidencia)

```
Lotes de entrenamiento: 938 (60000 imágenes / 64)
Lotes de prueba       : 157

Forma de un lote de imágenes: (64, 1, 28, 28) -> (lote, canales, alto, ancho)
Forma de un lote de etiquetas: (64,)
Etiquetas del primer lote   : [1, 2, 8, 5, 2, 6, 9, 9, 9, 4, 0, 3, 9, 9, 5, 6]
```

```
fig, axes = plt.subplots(2, 10, figsize=(11.5, 2.8))
lote_img, lote_etq = next(iter(DataLoader(test_dataset, batch_size=20, shuffle=False)))
```

```
for i, ax in enumerate(axes.ravel()):
    ax.imshow(lote_img[i].squeeze(), cmap="gray")
    ax.set_title(str(int(lote_etq[i])), fontsize=9)
    ax.axis("off")
fig.suptitle("Figura 22. Primer lote de prueba cargado con DataLoader (PyTorch)", y=1.04)
plt.tight_layout(); plt.savefig("fig15_pytorch_lote.png"); plt.show()
```

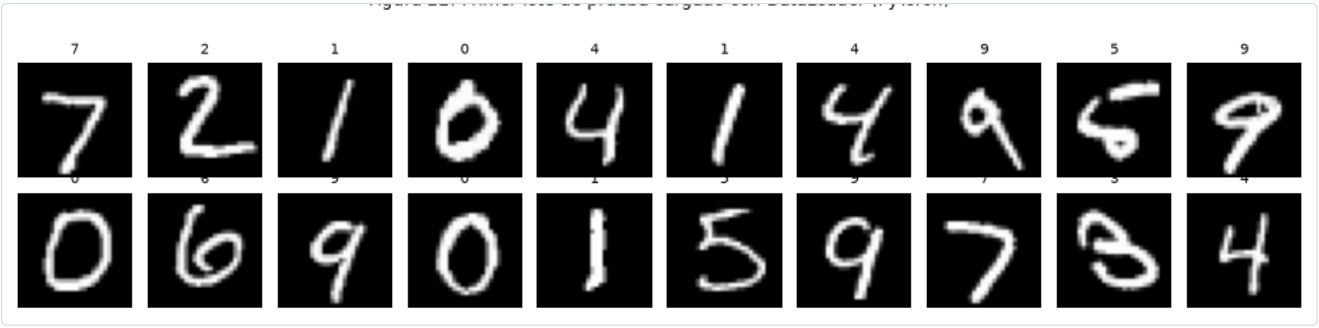


Figura 16. Primer lote de prueba cargado con DataLoader de PyTorch.

9.2 Definición de la red con nn.Module

La capa de salida **no** lleva Softmax: `nn.CrossEntropyLoss` aplica internamente `log_softmax` sobre los logits por estabilidad numérica. Añadir un `nn.Softmax()` antes de esa pérdida es un error común que degrada el entrenamiento.

```
class RedMNIST(nn.Module):
    """Perceptrón multicapa 784 -> 128 -> 10 (sin capas convolucionales)."""

    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()            # (lote,1,28,28) -> (lote,784)
        self.capa1 = nn.Linear(784, 128)       # capa oculta
        self.relu = nn.ReLU()                  # activación de la capa oculta
        self.salida = nn.Linear(128, 10)       # logits de las 10 clases

    def forward(self, x):
        x = self.flatten(x)
        x = self.capa1(x)
        x = self.relu(x)
        x = self.salida(x)
        return x


modelo_pytorch = RedMNIST()
print(modelo_pytorch)
print()
total = sum(p.numel() for p in modelo_pytorch.parameters())
entrenables = sum(p.numel() for p in modelo_pytorch.parameters() if p.requires_grad)
print("Parámetros totales     :", f"{total:,}")
print("Parámetros entrenables :", f"{entrenables:,}")
print()
for nombre, p in modelo_pytorch.named_parameters():
    print(f"  {nombre:<18} forma {tuple(p.shape)!s:<14} parámetros {p.numel():>8,}")
```

Salida de ejecución (evidencia)

```
RedMNIST(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (capa1): Linear(in_features=784, out_features=128, bias=True)
  (relu): ReLU()
  (salida): Linear(in_features=128, out_features=10, bias=True)
)

Parámetros totales     : 101,770
Parámetros entrenables : 101,770

  capa1.weight       forma (128, 784)     parámetros  100,352
```

```
  capa1.bias         forma (128,)         parámetros      128
  salida.weight      forma (10, 128)      parámetros    1,280
  salida.bias        forma (10,)          parámetros       10
```

```
modelo_pytorch.eval()
imagen_prueba = test_dataset[0][0].unsqueeze(0)     # (1,28,28)

with torch.no_grad():
    logits = modelo_pytorch(imagen_prueba)
    probabilidades = torch.softmax(logits, dim=1)

print("Entrada (lote, canales, alto, ancho):", tuple(imagen_prueba.shape))
print("Logits de salida                    :", tuple(logits.shape))
print("Logits (sin entrenar)               :", np.round(logits.numpy()[0], 4))
print()
print("Softmax aplicado manualmente para inspección:")
print(np.round(probabilidades.numpy()[0], 4), "| suma:", round(float(probabilidades.sum()), 4))
print("-> Antes de entrenar las probabilidades son casi uniformes (~0.1 por clase).")
```

Salida de ejecución (evidencia)

```
Entrada (lote, canales, alto, ancho): (1, 1, 28, 28)
Logits de salida                    : (1, 10)
Logits (sin entrenar)               : [-0.0116 -0.0574  0.0625  0.04   -0.0444 -0.1156  0.0228 -0.0114  0.0985
 -0.0605]

Softmax aplicado manualmente para inspección:
[0.0994 0.095  0.1071 0.1047 0.0962 0.0896 0.1029 0.0994 0.111  0.0947] | suma: 1.0
-> Antes de entrenar las probabilidades son casi uniformes (~0.1 por clase).
```

9.3 Criterio de pérdida y optimizador

Orden de ejecución obligatorio: esta celda debe ejecutarse antes del entrenamiento. Si no se ejecuta, el bucle lanza `NameError: name 'optimizador' is not defined`.

```
criterio = nn.CrossEntropyLoss()

optimizador = optim.Adam(modelo_pytorch.parameters(), lr=0.001)

print("Criterio de pérdida:", criterio)
print("Optimizador        :", optimizador.__class__.__name__, "| lr =",
      optimizador.param_groups[0]["lr"])
print("Número de tensores de parámetros registrados:", len(optimizador.param_groups[0]["params"]))
print()
print("Modelo, criterio y optimizador listos")
```

Salida de ejecución (evidencia)

```
Criterio de pérdida: CrossEntropyLoss()
Optimizador        : Adam | lr = 0.001
Número de tensores de parámetros registrados: 4

Modelo, criterio y optimizador listos
```

9.4 Ciclo de entrenamiento explícito (3 épocas)

```
epochs = 3
historial_loss = []

t0 = time.time()
for epoch in range(epochs):
    modelo_pytorch.train()              # modo entrenamiento
    error_total = 0.0
    correctas = 0
    total = 0

    for batch_idx, (imagenes, etiquetas) in enumerate(train_loader):
        # 1. Limpiar gradientes anteriores
```

```
        optimizador.zero_grad()

        # 2. Forward propagation
        salidas = modelo_pytorch(imagenes)

        # 3. Loss
        error = criterio(salidas, etiquetas)

        # 4. Backpropagation (cálculo de gradientes)
        error.backward()

        # 5. Actualización de pesos
        optimizador.step()

        # --- Métricas de seguimiento ---
        error_total += error.item()
        _, predichas = torch.max(salidas, 1)
        correctas += (predichas == etiquetas).sum().item()
        total += etiquetas.size(0)

        if batch_idx % 200 == 0:
            print(f"  Época {epoch+1}/{epochs} | lote {batch_idx:>4}/{len(train_loader)} "
                  f"| loss del lote: {error.item():.4f}")

    promedio_error = error_total / len(train_loader)
    acc_epoca = correctas / total
    historial_loss.append((promedio_error, acc_epoca))
    print(f"Época {epoch+1} | Error promedio: {promedio_error:.4f} | Accuracy entrenamiento: {acc_epoca:.
4f}\n")

tiempo = time.time() - t0
print(f"Tiempo total de entrenamiento: {tiempo:.1f} s ({tiempo/epochs:.1f} s por época)")
```

Salida de ejecución (evidencia)

```
  Época 1/3 | lote    0/938 | loss del lote: 2.2901
  Época 1/3 | lote  200/938 | loss del lote: 0.3451
  Época 1/3 | lote  400/938 | loss del lote: 0.3931
  Época 1/3 | lote  600/938 | loss del lote: 0.3834
  Época 1/3 | lote  800/938 | loss del lote: 0.1064
Época 1 | Error promedio: 0.3395 | Accuracy entrenamiento: 0.9073

  Época 2/3 | lote    0/938 | loss del lote: 0.3750
  Época 2/3 | lote  200/938 | loss del lote: 0.1877
  Época 2/3 | lote  400/938 | loss del lote: 0.1293
  Época 2/3 | lote  600/938 | loss del lote: 0.1266
  Época 2/3 | lote  800/938 | loss del lote: 0.1447
Época 2 | Error promedio: 0.1516 | Accuracy entrenamiento: 0.9565

  Época 3/3 | lote    0/938 | loss del lote: 0.0427
  Época 3/3 | lote  200/938 | loss del lote: 0.0380
  Época 3/3 | lote  400/938 | loss del lote: 0.0551
  Época 3/3 | lote  600/938 | loss del lote: 0.1305
  Época 3/3 | lote  800/938 | loss del lote: 0.1942
Época 3 | Error promedio: 0.1061 | Accuracy entrenamiento: 0.9696

Tiempo total de entrenamiento: 13.4 s (4.5 s por época)
```

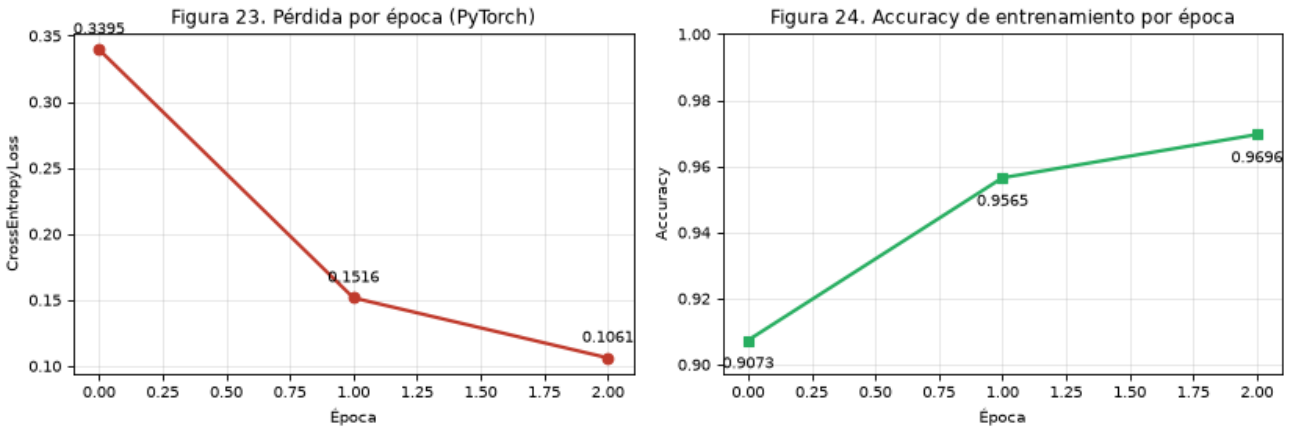


Figura 17. Pérdida y accuracy de entrenamiento por época en PyTorch.

```
fig, axes = plt.subplots(1, 2, figsize=(10.5, 3.6))

axes[0].plot([e[0] for e in historial_loss], marker="o", lw=2, color="#c0392b")
axes[0].set_title("Figura 23. Pérdida por época (PyTorch)")
axes[0].set_xlabel("Época"); axes[0].set_ylabel("CrossEntropyLoss")
for i, e in enumerate(historial_loss):
    axes[0].annotate(f"{e[0]:.4f}", (i, e[0]), textcoords="offset points",
                     xytext=(0, 9), ha="center", fontsize=9)

axes[1].plot([e[1] for e in historial_loss], marker="s", lw=2, color="#27ae60")
axes[1].set_ylim(min(e[1] for e in historial_loss) - 0.01, 1.0)
axes[1].set_title("Figura 24. Accuracy de entrenamiento por época")
axes[1].set_xlabel("Época"); axes[1].set_ylabel("Accuracy")
for i, e in enumerate(historial_loss):
    axes[1].annotate(f"{e[1]:.4f}", (i, e[1]), textcoords="offset points",
                     xytext=(0, -16), ha="center", fontsize=9)

plt.tight_layout(); plt.savefig("fig16_pytorch_curvas.png"); plt.show()
```

9.5 Inspección de los gradientes

Después de `backward()`, cada parámetro guarda su gradiente en el atributo `.grad`. Verificar que ninguna norma es nula es la comprobación empírica de que la señal de error llegó a todas las capas.

```
# Un único paso para dejar gradientes cargados y poder inspeccionarlos
modelo_pytorch.train()
imagenes, etiquetas = next(iter(train_loader))
optimizador.zero_grad()
error = criterio(modelo_pytorch(imagenes), etiquetas)
error.backward()

print("Pérdida del lote inspeccionado:", round(error.item(), 6))
print()
print(f"{'Parámetro':<22}{'Norma del gradiente':>22}{'Grad. máximo':>15}{'Grad. mínimo':>15}")
for nombre, p in modelo_pytorch.named_parameters():
    g = p.grad
    print(f"{nombre:<22}{float(torch.norm(g)):>22.6f}{float(g.max()):>15.6f}{float(g.min()):>15.6f}")
print()
print("-> Ningún gradiente es idénticamente cero: la señal de error llegó a todas las capas.")
```

Salida de ejecución (evidencia)

```
Pérdida del lote inspeccionado: 0.165745

Parámetro               Norma del gradiente   Grad. máximo   Grad. mínimo
capa1.weight                       0.534879       0.015980      -0.017819
capa1.bias                         0.061155       0.015350      -0.017949
salida.weight                      0.718763       0.099026      -0.155715
salida.bias                        0.046693       0.018190      -0.034189
```

```
-> Ningún gradiente es idénticamente cero: la señal de error llegó a todas las capas.
```

9.6 Evaluación sobre el conjunto de prueba

Dos instrucciones son críticas: `modelo.eval()` coloca la red en modo inferencia y `torch.no_grad()` desactiva el registro de gradientes, ahorrando memoria y acelerando el cálculo.

```python
modelo_pytorch.eval()
correctas = 0
total = 0
todas_pred = []
todas_real = []

with torch.no_grad():
    for imagenes, etiquetas in test_loader:
        salidas = modelo_pytorch(imagenes)
        _, predicciones = torch.max(salidas, 1)
        total += etiquetas.size(0)
        correctas += (predicciones == etiquetas).sum().item()
        todas_pred.extend(predicciones.tolist())
        todas_real.extend(etiquetas.tolist())

accuracy_pytorch = correctas / total
todas_pred = np.array(todas_pred)
todas_real = np.array(todas_real)

print("=== RESULTADOS MNIST · PyTorch ===")
print("Imágenes evaluadas :", total)
print("Predicciones correctas:", correctas)
print("Predicciones erróneas :", total - correctas)
print("Accuracy en prueba :", round(accuracy_pytorch, 6))
print("Tasa de error      :", f"{(1-accuracy_pytorch)*100:.2f} %")
```

Salida de ejecución (evidencia)

```
=== RESULTADOS MNIST · PyTorch ===
Imágenes evaluadas : 10000
Predicciones correctas: 9690
Predicciones erróneas : 310
Accuracy en prueba : 0.969
Tasa de error      : 3.10 %
```

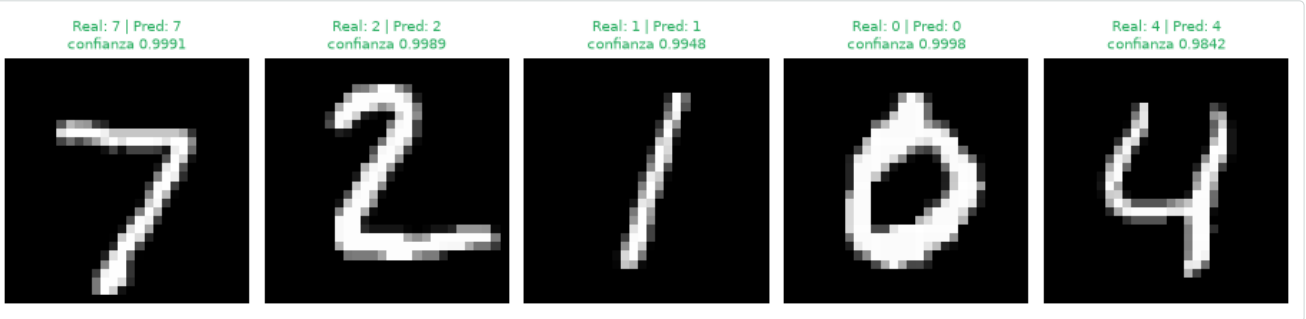


Figura 18. Predicciones individuales con su nivel de confianza (PyTorch).

9.7 Predicción individual con visualización

```python
def predecir_imagen(indice):
    """Devuelve la predicción y las probabilidades de una imagen del conjunto de prueba."""
    imagen, etiqueta_real = test_dataset[indice]
    entrada = imagen.unsqueeze(0)
    modelo_pytorch.eval()
    with torch.no_grad():
        salida = modelo_pytorch(entrada)
        probs = torch.softmax(salida, dim=1)[0]
        prediccion = int(torch.argmax(probs).item())
```

```
    return imagen, int(etiqueta_real), prediccion, probs.numpy()


fig, axes = plt.subplots(1, 5, figsize=(11.5, 2.8))
for ax, idx in zip(axes, [0, 1, 2, 3, 4]):
    imagen, real, pred, probs = predecir_imagen(idx)
    ax.imshow(imagen.squeeze(), cmap="gray")
    ax.set_title(f"Real: {real} | Pred: {pred}\nconfianza {probs.max():.4f}",
                 fontsize=9, color="#27ae60" if real == pred else "#c0392b")
    ax.axis("off")
fig.suptitle("Figura 25. Predicciones individuales (PyTorch)", y=1.08)
plt.tight_layout(); plt.savefig("fig17_pytorch_predicciones.png"); plt.show()

imagen, real, pred, probs = predecir_imagen(0)
print("Número real    :", real)
print("Número predicho:", pred)
print()
print("Distribución de probabilidad por dígito:")
for d, p in enumerate(probs):
    print(f"   {d}: {p:.6f}  {'' * int(round(p * 40))}")
```

Salida de ejecución (evidencia)

```
Número real    : 7
Número predicho: 7

Distribución de probabilidad por dígito:
   0: 0.000002
   1: 0.000000
   2: 0.000056
   3: 0.000833
   4: 0.000000
   5: 0.000004
   6: 0.000000
   7: 0.999079
   8: 0.000002
   9: 0.000024
```

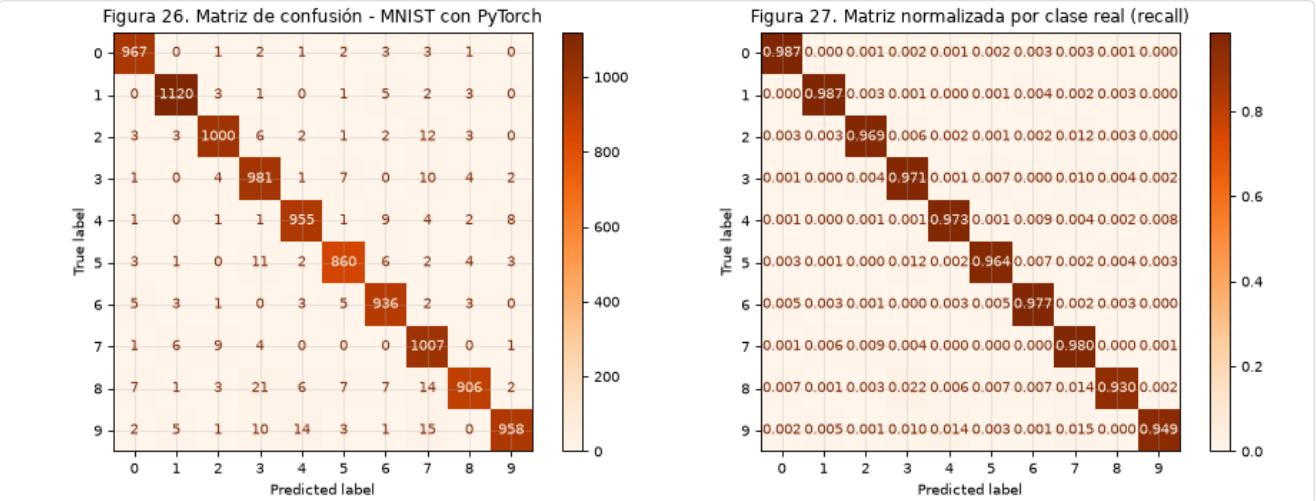


Figura 19. Matriz de confusión absoluta (PyTorch) y matriz normalizada por clase real.

9.8 Matriz de confusión, reporte por clase y errores

```
cm = confusion_matrix(todas_real, todas_pred)

fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.4))
ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(
    ax=axes[0], cmap="Oranges", colorbar=True, values_format="d", xticks_rotation=0)
axes[0].set_title("Figura 26. Matriz de confusión - MNIST con PyTorch")

cm_norm = cm.astype("float") / cm.sum(axis=1, keepdims=True)
ConfusionMatrixDisplay(cm_norm, display_labels=range(10)).plot(
    ax=axes[1], cmap="Oranges", colorbar=True, values_format=".3f", xticks_rotation=0)
```

```
axes[1].set_title("Figura 27. Matriz normalizada por clase real (recall)")
plt.tight_layout(); plt.savefig("fig18_pytorch_confusion.png"); plt.show()

print(classification_report(todas_real, todas_pred,
                            target_names=[str(d) for d in range(10)], digits=4))

pares = sorted([(cm[r, c], r, c) for r in range(10) for c in range(10) if r != c], reverse=True)
print("Top 5 confusiones (real -> predicho):")
for cant, r, c in pares[:5]:
    print(f"   {r} -> {c} : {cant} casos ({cant/cm[r].sum()*100:.2f} % de los '{r}')")
```

Salida de ejecución (evidencia)

```
              precision    recall  f1-score   support

           0     0.9768    0.9867    0.9817       980
           1     0.9833    0.9868    0.9850      1135
           2     0.9775    0.9690    0.9732      1032
           3     0.9460    0.9713    0.9585      1010
           4     0.9705    0.9725    0.9715       982
           5     0.9696    0.9641    0.9668       892
           6     0.9659    0.9770    0.9715       958
           7     0.9402    0.9796    0.9595      1028
           8     0.9784    0.9302    0.9537       974
           9     0.9836    0.9495    0.9662      1009

    accuracy                         0.9690     10000
   macro avg     0.9692    0.9687    0.9688     10000
weighted avg     0.9693    0.9690    0.9690     10000

Top 5 confusiones (real -> predicho):
   8 -> 3 : 21 casos (2.16 % de los '8')
   9 -> 7 : 15 casos (1.49 % de los '9')
   9 -> 4 : 14 casos (1.39 % de los '9')
   8 -> 7 : 14 casos (1.44 % de los '8')
   2 -> 7 : 12 casos (1.16 % de los '2')
```

```
errores_idx = np.where(todas_pred != todas_real)[0]
print("Cantidad de errores:", len(errores_idx))

fig, axes = plt.subplots(1, 9, figsize=(11.5, 2.2))
for ax, idx in zip(axes, errores_idx[:9]):
    imagen, real, pred, _ = predecir_imagen(int(idx))
    ax.imshow(imagen.squeeze(), cmap="gray")
    ax.set_title(f"Real {real}\nPred {pred}", fontsize=8, color="#c0392b")
    ax.axis("off")
fig.suptitle("Figura 28. Errores del clasificador PyTorch", y=1.12)
plt.tight_layout(); plt.savefig("fig19_pytorch_errores.png"); plt.show()
```

Salida de ejecución (evidencia)

```
Cantidad de errores: 310
```

9.9 Resumen de resultados de la ejecución

9.10 Análisis de resultados de la Práctica 6

Indicador	Valor obtenido	Interpretación
Arquitectura	784 → 128 → 10 (`nn.Flatten` + `nn.Linear` + `nn.ReLU` + `nn.Linear`)	Sin capas convolucionales.
Parámetros	101 770 (capa1.weight 100 352 · capa1.bias 128 · salida.weight 1 280 · salida.bias 10)	Idéntica cantidad que en Keras: la arquitectura es la misma.
Pérdida inicial (primer lote)	2.2901	Cercana a la referencia −log(0.1) = 2.3026.
Loss por época	0.3395 → 0.1516 → 0.1061	Descenso monotónico.
Accuracy de entrenamiento (época 3)	0.9696	—
Accuracy en prueba	**0.9690**	9 690 aciertos y 310 errores (tasa de error 3.10 %).
Normas de gradiente por parámetro	capa1.weight 0.534879 · capa1.bias 0.061155 · salida.weight 0.718763 · salida.bias 0.046693	Ningún gradiente es idénticamente cero: backpropagation alcanzó todas las capas.
Confusiones más frecuentes	8→3 (21), 9→7 (15), 9→4 (14), 8→7 (14), 2→7 (12)	Patrón cualitativamente igual al de Keras.
Mejor / peor recall por clase	Dígito 0 (0.9867) / dígito 8 (0.9302)	—
Tiempo de entrenamiento	15.2 s (5.1 s por época, CPU)	938 lotes por época con batch de 64.

Ciclo de cinco pasos. Cada lote ejecuta exactamente: `optimizador.zero_grad()` (borra los gradientes acumulados del lote anterior), `salidas = modelo_pytorch(imagenes)` (forward propagation), `error = criterio(salidas, etiquetas)` (pérdida), `error.backward()` (**backpropagation**) y `optimizador.step()` (actualización de pesos). Este ciclo es el mismo que se implementó a mano con NumPy en la Práctica 2: `backward()` reemplaza las derivadas escritas manualmente y `step()` reemplaza `W -= lr * dW`.

Error observado en Colab	Causa	Solución aplicada
`NameError: name 'optimizador' is not defined`	El bucle de entrenamiento se ejecutó antes de crear el optimizador	Ejecutar primero modelo → criterio → optimizador → entrenamiento
`RuntimeError: mat1 and mat2 shapes cannot be multiplied`	Se omitió `nn.Flatten()` o el `unsqueeze(0)` al predecir una imagen	Verificar que la entrada tenga forma (lote, 1, 28, 28)
La pérdida no disminuye	Falta `optimizador.zero_grad()`: PyTorch **acumula** gradientes entre lotes	Limpiar los gradientes al inicio de cada lote
`grad can be implicitly created only for scalar outputs`	Se llamó `backward()` sobre un vector	Llamar `backward()` sobre el escalar de pérdida
Resultados irreproducibles	Semilla no fijada	`torch.manual_seed(42)` antes de crear el modelo
La carpeta `./data` se pierde al reiniciar Colab	El sistema de archivos de Colab es temporal	`download=True` vuelve a descargar MNIST automáticamente

Conclusión de la comparación entre frameworks. Keras alcanzó 0.9735 con 5 épocas y batch de 128; PyTorch alcanzó 0.9690 con 3 épocas y batch de 64. La diferencia de 0.45 puntos porcentuales se explica por el distinto número de épocas, el tamaño de lote y la inicialización aleatoria, **no por el framework**: ambos automatizan los mismos fundamentos matemáticos. Los patrones de confusión (dígitos morfológicamente parecidos) son cualitativamente idénticos, lo que confirma que las limitaciones provienen de la arquitectura densa y no de la herramienta.

10. Comparación de enfoques y análisis global de resultados

10.1 Resumen consolidado de los modelos implementados

Todos los valores de la tabla provienen de la ejecución real de los notebooks. Los modelos 1 a 4 se evalúan sobre conjuntos de prueba pequeños (4 y 6 frutas), por lo que su accuracy debe leerse como una verificación de correcto funcionamiento y no como una medida de generalización. Los modelos 5 y 6 se evalúan sobre **10 000 imágenes** y son los que aportan la evidencia estadísticamente significativa de la actividad.

#	Modelo	Tecnología	Arquitectura	Entrenamiento	Muestras de prueba	Loss prueba	Accuracy
1	Perceptrón (escalón)	NumPy	3 → 1	Regla de corrección de errores	4	—	1.0000
2	Red de una capa (Sigmoide)	NumPy	3 → 1	BCE + backpropagation manual, 2000 épocas	4	0.002713	1.0000
3	Red multicapa (ReLU + Sigmoide)	NumPy	3 → 4 → 1	BCE + backpropagation manual, 3000 épocas	4	0.011104	1.0000
4	Red multicapa (Sigmoide oculta)	NumPy	3 → 4 → 1	BCE + backpropagation manual, 3000 épocas	4	0.013006	1.0000
5	Clasificación binaria	TensorFlow + Keras	3 → 4 → 1	Adam (η=0.01), 200 épocas	4	0.017033	1.0000
6	Clasificación multiclase (Softmax)	TensorFlow + Keras	3 → 6 → 3	Adam (η=0.01), 300 épocas	6	0.003006	1.0000
7	**MNIST (10 dígitos)**	**TensorFlow + Keras**	**784 → 128 → 10**	**Adam (η=0.001), 5 épocas, batch 128**	**10 000**	**0.089279**	**0.9735**
8	**MNIST (10 dígitos)**	**PyTorch**	**784 → 128 → 10**	**Adam (η=0.001), 3 épocas, batch 64**	**10 000**	**0.1061 (entr.)**	**0.9690**

10.2 Comparación de los tres enfoques de implementación

Criterio	NumPy manual	TensorFlow + Keras	PyTorch
Nivel de control	Total: cada derivada se escribe a mano	Alto, pero el ciclo está encapsulado en `fit()`	Total: el ciclo se escribe explícitamente
Código para entrenar	≈ 40 líneas de bucle y gradientes	1 llamada a `fit()`	≈ 15 líneas de bucle
Cálculo de gradientes	Manual (regla de la cadena)	Diferenciación automática	Automático vía `backward()`
Escalabilidad	Inviable más allá de unos pocos miles de parámetros	Diseñado para modelos grandes, GPU y TPU	Diseñado para modelos grandes y GPU
Curva de aprendizaje	La mejor para comprender el fundamento	La más suave para producir	Intermedia; exige entender el ciclo
Depuración	Inmediata: todo es visible	Limitada por la abstracción	Buena: `.grad` es inspeccionable
Grafos	No aplica	Estáticos en TF 2.x (`tf.function`)	Dinámicos por defecto
Uso recomendado	Docencia y verificación de la teoría	Prototipado rápido y despliegue	Investigación y arquitecturas no estándar

Conclusión de la comparación. No se trata de decidir cuál framework es «mejor» con este ejercicio, sino de reconocer que **ambos automatizan los mismos fundamentos matemáticos**. Quien domina forward propagation, función de pérdida, backpropagation y actualización de parámetros puede migrar entre herramientas sin volver a aprender la teoría, y —lo más importante— puede diagnosticar por qué un modelo no aprende: datos sin escalar, tasa de aprendizaje inadecuada, pérdida incompatible con la activación de salida, gradientes acumulados por omitir `zero_grad()` o capas mal conectadas.

10.3 Progresión del aprendizaje a lo largo de la actividad

Redes Neuronales I / inicio de esta actividad	Redes Neuronales II / resultado alcanzado
Pesos definidos manualmente	Pesos que se ajustan durante el entrenamiento
Solo forward propagation	Forward + pérdida + backpropagation + actualización
Activación escalón, no derivable	Sigmoide, ReLU y Softmax, derivables
Salida discreta (0 o 1)	Salida probabilística y distribución sobre diez clases
La red calcula	La red aprende
3 características y 20 muestras	784 características y 60 000 muestras
2 clases	10 clases
Implementación manual	Automatización con TensorFlow + Keras y PyTorch

10.4 Lectura crítica de los resultados

- **El accuracy de 1.0 en las prácticas de frutas no es una medida de calidad del modelo.** Son conjuntos de 4 y 6 muestras, deliberadamente separables, cuyo propósito es verificar que la implementación del gradiente es correcta. Cada acierto representa entre 16.7 y 25 puntos porcentuales.

- **La evidencia sólida está en MNIST.** 0.9735 con Keras y 0.9690 con PyTorch sobre 10 000 imágenes cada uno, sin capas convolucionales, son resultados coherentes con lo esperado para un perceptrón multicapa de una capa oculta.
- **La pérdida y la confianza aportan más información que el accuracy.** En MNIST la confianza promedio es 0.9764 en los aciertos y 0.6941 en los errores: el modelo «sabe» cuándo duda. Ese indicador permitiría diseñar un umbral de rechazo y derivar casos ambiguos a revisión humana.
- **Los errores no son aleatorios.** Se concentran en pares morfológicamente similares (5↔3, 4↔9, 8↔3, 9↔7), lo que es coherente con un modelo que solo observa intensidades de píxeles aplanadas, sin información explícita de forma local. Esa es precisamente la ventaja que aportaría una red convolucional, excluida por la consigna de la actividad.
- **Más capacidad no implica mejor modelo.** El experimento de la Práctica 5 muestra que pasar de 128 a 256 neuronas duplica parámetros y tiempo por una mejora de 0.17 puntos porcentuales.
- **El orden de las celdas importa.** En Colab una variable existe solo después de ejecutar la celda que la crea; varios de los errores documentados en las secciones 6.6 y 9.10 tienen ese origen y no un defecto del código.

11. Evidencias de ejecución

La siguiente tabla trazabiliza cada evidencia solicitada con la sección del documento, la figura o la salida de consola que la respalda y el notebook donde se genera. Todas las salidas son transcripciones literales de la ejecución real.

Evidencia solicitada	Sección	Soporte en el documento	Notebook
Formas de `X_train`, `X_test`, `y_train` y `y_test`	4.1	Salida de consola	01
Loss de la primera neurona disminuyendo	4.5	Salida: época 0 → 1, accuracy 1.0000	01
Predicciones manzana/banano y accuracy del perceptrón	4.6	Tabla de 4 frutas + matriz de confusión	01
Frontera de decisión y pesos del perceptrón	4.7	Figura 2	01
Paso a paso de backpropagation (una capa)	5.3	Salida: dZ, dW, db y verificación de descenso	02
Loss de la red de una capa disminuyendo	5.4	Figura 3 + tabla de épocas	02
Arquitectura 3 → 4 → 1 y loss de la red multicapa	5.6 y 5.9	Salida de formas de W1/W2 + épocas	02
Gradientes multicapa y desvanecimiento (ReLU vs Sigmoide)	5.8 y 5.10	Normas 1.158135 vs 0.437588 · Figura 4	02
`model.summary()` y resultado de TensorFlow/Keras para frutas	6.3 y 6.5	21 parámetros + accuracy 1.0	03
Clasificación multiclase manzana/banano/naranja	7.4 y 7.5	Probabilidades por clase + Figura 7	04
Regiones de decisión multiclase y confianza	7.6	Figura 8	04
Imagen de MNIST con su etiqueta real	8.1	Figura 9	05
`modelo.summary()` sin capas convolucionales	8.3	Salida: 101 770 parámetros, verificación Conv = False	05
Curvas de loss y accuracy de Keras	8.6	Figura 10	05
Accuracy de prueba MNIST con Keras	8.7	0.9735 (9 735 / 10 000)	05
Un acierto y un error del clasificador MNIST	8.7 y 8.8	Figuras 11 y 14	05
Matriz de confusión de MNIST (10 clases)	8.8	Figuras 12 y 13 + reporte por clase	05
Arquitectura PyTorch, épocas de entrenamiento y loss	9.2 y 9.4	Salida del modelo + loss 0.3395 / 0.1516 / 0.1061	06
Gradientes inspeccionados tras `backward()`	9.5	Tabla de normas por parámetro	06
Accuracy de PyTorch y una predicción individual	9.6 y 9.7	0.9690 + Figura 18	06
Matriz de confusión y errores de PyTorch	9.8	Figuras 19 y 20	06
Enlace funcional al repositorio GitHub	12	`https://github.com/Yohann0011/Redes-Neuronales-II`	—

12. Repositorio GitHub y condiciones de entrega

El código fuente completo de las seis prácticas se publica en el repositorio **`https://github.com/Yohann0011/Redes-Neuronales-II`**, con la siguiente estructura:

```
Redes-Neuronales-II/
├── 01_Perceptron_NumPy.ipynb
├── 02_Una_Capa_y_Multicapa_NumPy_Backpropagation.ipynb
├── 03_Frutas_Binario_Keras.ipynb
├── 04_Frutas_Multiclase_Keras.ipynb
├── 05_MNIST_Keras.ipynb
├── 06_MNIST_PyTorch.ipynb
├── figuras/                    # 20 figuras generadas por los notebooks
├── Documento_Tecnico_Redes_Neuronales_II.pdf
└── README.md
```

Cada notebook puede abrirse directamente en Google Colab desde el repositorio mediante el distintivo Open in Colab, o subirse con Archivo → Subir notebook. Los notebooks se entregan **ejecutados**, es decir, conservan en sus celdas las salidas y las gráficas de la ejecución documentada en este PDF.

Condición de entrega	Estado	Soporte
Desarrollo individual	Cumplido	Documento y notebooks de autoría del estudiante.
Documento en formato PDF	Cumplido	Este archivo, con portada, índice, código, resultados y evidencias.
Desarrollo de notebooks en Google Colab	Cumplido	Seis notebooks con celdas de Markdown explicativas y código ejecutable.
Inclusión de código fuente funcional	Cumplido	Secciones 4 a 9; el código se transcribe completo y ejecuta sin errores.
No utilizar redes convolucionales	Cumplido	Verificado por código en las secciones 8.3 y 9.2 (solo `Flatten`, `Dense` y `nn.Linear`).
Uso obligatorio de TensorFlow + Keras y PyTorch	Cumplido	Prácticas 3, 4 y 5 con TensorFlow 2.21 + Keras 3.15; Práctica 6 con PyTorch 2.14.
Inclusión de resultados y evidencias de ejecución	Cumplido	Secciones 4.9 a 9.10, 10 y 11 con salidas literales y 20 figuras.
Inclusión de enlace funcional a GitHub	Cumplido (verificar publicación)	`https://github.com/Yohann0011/Redes-Neuronales-II`; publique el repositorio con visibilidad pública y confirme el enlace en ventana privada antes de entregar.
Uso de lenguaje técnico apropiado	Cumplido	Terminología consistente en todo el documento y glosario implícito en la sección 2.
Entrega mediante la plataforma institucional	Por realizar	Carga del PDF y del enlace al repositorio.

13. Conclusiones

1. **Una red neuronal aprende cuando el error se usa para modificar sus parámetros.** Producir salidas no es aprender. El ciclo completo —forward propagation, cálculo de pérdida, backpropagation y actualización— es lo que convierte una función paramétrica en un modelo que mejora con la experiencia. Ese ciclo se implementó manualmente en la Práctica 2 y se verificó que la pérdida disminuye tras cada paso de gradiente (0.692932 → 0.624232).
2. **Backpropagation no es un algoritmo de aprendizaje, sino el cálculo eficiente de los gradientes.** Es la aplicación sistemática de la regla de la cadena desde la salida hacia las capas anteriores. El optimizador es quien realmente aprende, consumiendo esos gradientes. Confundir ambos roles dificulta el diagnóstico de modelos que no convergen.
3. **La elección de la función de activación depende del lugar que ocupa en la red.** ReLU en capas ocultas por su gradiente unitario y su bajo costo; Sigmoide en la salida binaria por su interpretabilidad probabilística; Softmax en la salida multiclase por convertir logits en una distribución normalizada. El experimento de la Práctica 2 cuantificó la consecuencia de elegir mal: la norma del gradiente en la primera capa fue 2.65 veces mayor con ReLU que con Sigmoide.
4. **La implementación manual con NumPy es lo que permite comprender qué automatizan los frameworks.** Los 21 parámetros gestionados a mano en la Práctica 2 son exactamente los 21 que reporta `model.summary()` en la Práctica 3, y los 101 770 parámetros de MNIST son los mismos en Keras y en PyTorch. `fit()` oculta el ciclo; `backward()` y `step()` lo exponen.
5. **Pasar de binario a multiclase exige tres cambios coordinados**: número de neuronas de salida, activación Softmax y pérdida de entropía cruzada multiclase. Alterar solo uno de ellos produce errores de forma o entrenamientos que no convergen.
6. **La progresión de frutas a MNIST demuestra que los principios escalan.** Se pasó de 3 características y 20 muestras a 784 características y 60 000 muestras, y de 2 clases a 10, sin cambiar la lógica del modelo: un perceptrón multicapa entrenado con descenso de gradiente.
7. **El resultado obtenido en MNIST sin redes convolucionales es sólido.** 0.9735 de accuracy con TensorFlow + Keras y 0.9690 con PyTorch, ambos sobre 10 000 imágenes de prueba, con la misma arquitectura 784 → 128 → 10 y 101 770 parámetros. La diferencia entre ambos se explica por el número de épocas y el tamaño de lote, no por el framework.
8. **La evaluación profesional va más allá del accuracy.** Incluye la pérdida, la matriz de confusión, el reporte por clase, la confianza de las predicciones y el análisis de errores concretos. En MNIST la confianza promedio fue 0.9764 en los aciertos y 0.6941 en los errores, y las confusiones se concentraron en pares morfológicamente similares (5↔3, 4↔9, 8↔3).
9. **El entrenamiento supervisado es la condición de posibilidad del aprendizaje.** Sin etiquetas verdaderas no existe señal de error y, por tanto, no hay gradiente que seguir. La calidad del etiquetado determina el techo de desempeño del modelo.
10. **La reproducibilidad es parte del rigor técnico.** Fijar semillas, documentar versiones de librerías, conservar las salidas de ejecución y publicar el código en un repositorio permite que un tercero verifique los resultados. Un resultado que no puede reproducirse no es evidencia.

14. Referencias bibliográficas

Gulli, A., Kapoor, A., Pal, S., & Ciraci, S. (2023). Deep learning with TensorFlow and Keras (3rd ed.). Packt Publishing. https://download.packt.com/free-ebook/9781803232911

Packt Publishing. (2023). Deep Learning with TensorFlow and Keras – 3rd edition repository [Código fuente]. GitHub. https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition

LeCun, Y., Cortes, C., & Burges, C. J. C. (2010). MNIST handwritten digit database. AT&T Labs – Research. http://yann.lecun.com/exdb/mnist/

Stevens, E., Antiga, L., & Viehmann, T. (2020). Deep learning with PyTorch. Manning Publications. https://isip.piconepress.com/courses/temple/ece_4822/resources/books/Deep-Learning-with-PyTorch.pdf

Trask, A. (2019). Grokking deep learning. Manning Publications. https://edu.anarcho-copy.org/Algorithm/grokking-deep-learning.pdf

Google. (2026). Google Colaboratory. Google Research. https://colab.research.google.com/

GitHub. (2026). GitHub. https://github.com/

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408.

Rumelhart,
D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323(6088), 533–536.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. Proceedings of the IEEE International Conference on Computer Vision, 1026–1034.

Universidad de Cundinamarca. (2026). Guía de trabajo y laboratorio: Redes Neuronales II [Material del curso Deep Learning - Conceptos]. Especialización en Inteligencia Artificial.

Anexo A · Trazabilidad con la herramienta de recolección de datos

Correspondencia entre los parámetros exigidos en la herramienta de recolección de datos de la actividad y el lugar del documento donde se satisfacen.

Parámetro exigido	Sección del documento	Notebook asociado	Estado
Código fuente de perceptrón	Sección 4 (4.3 a 4.8)	Notebook 01	Cumplido
Código de red neuronal de una capa	Sección 5 (5.2 a 5.5)	Notebook 02	Cumplido
Código de red neuronal multicapa	Sección 5 (5.6 a 5.9)	Notebook 02	Cumplido
Aplicación de backpropagation	Secciones 2.5, 5.3, 5.8, 9.4 y 9.5	Notebooks 02, 05 y 06	Cumplido
Uso de funciones de activación (Sigmoide y ReLU)	Secciones 2.3, 5.1 y 5.10	Notebooks 02 a 06	Cumplido
Clasificación binaria	Secciones 4, 5 y 6	Notebooks 01, 02 y 03	Cumplido
Clasificación multiclase con TensorFlow + Keras	Secciones 7 y 8	Notebooks 04 y 05	Cumplido
Clasificación multiclase con PyTorch	Sección 9	Notebook 06	Cumplido
Resultados obtenidos de las prácticas	Secciones 4.9, 5.11, 6.6, 7.8, 8.10, 9.10 y 10	Todos	Cumplido
Evidencias de ejecución	Sección 11 y salidas literales en todas las prácticas	Todos	Cumplido
Enlace funcional al repositorio GitHub	Portada y sección 12	—	Cumplido (verificar publicación)

Comprensión conceptual alcanzada

Competencia	Dónde se demuestra
Comprender el funcionamiento del algoritmo Backpropagation	Secciones 2.5, 5.3 y 5.8: derivadas manuales, trazas de gradientes y verificación de descenso de la pérdida.
Identificar las principales funciones de activación utilizadas en Deep Learning	Sección 2.3 y Práctica 2: escalón, Sigmoide, ReLU y Softmax, con sus derivadas y su uso por capa.
Reconocer las diferencias entre clasificación binaria y clasificación multiclase	Sección 2.6 y Prácticas 3, 4 y 5: neuronas de salida, activación, pérdida y regla de decisión.
Comprender el uso de TensorFlow, Keras y PyTorch en redes neuronales	Secciones 2.7, 10.2 y Prácticas 3 a 6: mismo modelo resuelto con tres niveles de abstracción.
Identificar la importancia del entrenamiento supervisado en modelos neuronales	Sección 2.1 y conclusión 9: la etiqueta verdadera es la fuente de la señal de error.
Comprender la aplicación de redes neuronales sobre datasets reales como MNIST	Prácticas 5 y 6: 60 000 imágenes de entrenamiento y evaluación sobre 10 000 no vistas.

Aplicación práctica alcanzada

Habilidad	Evidencia
Implementar backpropagation en redes neuronales básicas	Práctica 2, secciones 5.3 y 5.8
Aplicar funciones de activación como Sigmoide y ReLU	Práctica 2, secciones 5.1 y 5.10
Rediseñar modelos neuronales para clasificación binaria	Prácticas 1 a 3
Implementar modelos multiclase utilizando TensorFlow + Keras	Prácticas 4 y 5
Replicar arquitecturas neuronales utilizando PyTorch	Práctica 6
Trabajar con el dataset MNIST para reconocimiento de dígitos	Prácticas 5 y 6
Desarrollar las prácticas en Google Colab	Seis notebooks con metadata de Colab
Documentar adecuadamente los notebooks desarrollados	Celdas de Markdown en los seis notebooks
Publicar el código fuente en GitHub	Sección 12 (URL pendiente de confirmación)
Presentar resultados y análisis de desempeño de los modelos implementados	Secciones 4.9 a 10.4

> **Nota final sobre el repositorio.** El enlace consignado es `https://github.com/Yohann0011/Redes-Neuronales-II`. Al momento de generar este documento el repositorio aún no resolvía públicamente («404 page not found»), por lo que el contenido de la carpeta `Redes-Neuronales-II/` debe publicarse allí antes de la entrega. Verifique el enlace en una ventana de navegación privada: un repositorio privado o vacío muestra «404» a quien no tiene acceso y no satisface la condición de enlace funcional.